

TRƯỜNG ĐẠI HỌC THỦ DẦU MỘT
VIỆN CÔNG NGHỆ SỐ



ĐỒ ÁN CHUYÊN NGÀNH

XÂY DỰNG WEB QUẢN LÝ HỌC TẬP TRỰC TUYẾN LMS THÔNG MINH

Sinh viên thực hiện:

1. Lê Vũ Anh

Mã số sinh viên: 2224801030388

Lớp: D22KTPM01

2. Nguyễn Thanh Duy

Mã số sinh viên: 2224801030170

Lớp: D22KTPM01

Giảng viên Hướng dẫn: **TS. Huỳnh Nguyễn Thành Luân**

TP. Hồ Chí Minh - 2026

MỤC LỤC

MỤC LỤC	i
DANH MỤC HÌNH	iv
DANH MỤC BẢNG	v
DANH SÁCH CÁC KÝ TỰ, CHỮ VIẾT TẮT	vi
MỞ ĐẦU	1
CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG LMS.....	5
1.1. Giới thiệu về Learning Management System (LMS).....	5
1.2. Vai trò của LMS trong giáo dục số.....	6
1.3. Các Mô hình kiến trúc phổ biến	6
1.3.1. Monolithic Architecture.....	7
1.3.2. Microservices Architecture.....	8
1.4. So sánh các mô hình kiến trúc	8
1.5. Định hướng xây dựng Intelligent LMS	9
CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG.....	11
2.1. Mô tả bài toán.....	11
2.2. Các tác nhân hệ thống.....	11
2.3. Chức năng hệ thống	12
2.4. Yêu cầu phi chức năng.....	13
2.5. Phân tích rủi ro hệ thống.....	13
2.5.1. Rủi ro về tính nhất quán dữ liệu (Eventual Consistency)	13
2.5.2. Rủi ro về vận hành và giám sát (Operational Complexity)	14
2.5.3. Rủi ro tại tầng API Gateway (Security & SPOF)	14
2.5.4. Rủi ro về độ chính xác của AI Advisor (AI Accuracy)	14
CHƯƠNG 3. THIẾT KẾ HỆ THỐNG.....	15
3.1. Kiến trúc tổng thể hệ thống.....	15
3.2. Thiết kế API Gateway.....	16
3.3. Thiết kế các Backend Services	17
3.3.1. Auth Service (Dịch vụ xác thực)	17
3.3.2. User Service (Dịch vụ người dùng)	17
3.3.3. Course Service (Dịch vụ khóa học)	17
3.3.4. Progress Service (Dịch vụ tiến độ)	17
3.4. Use Case Diagram	18
3.5. Đặc tả usecase	20
3.5.1. Use Case: UC001 – Đăng ký tài khoản	20
3.5.2. Use Case: UC002 – Xem Dashboard và Tiến độ học tập.....	21
3.5.3. Use Case: UC003 – Quản lý nội dung bài giảng.....	21
3.5.4. Use Case: UC004 – Nhận tư vấn lộ trình học tập (AI Advisor).....	22
3.5.5. Use Case: UC005 – Quản lý tài khoản và Phân quyền	22

3.6. Thiết kế cơ sở dữ liệu	23
3.6.1. Bảng Enrollments (Quản lý đăng ký khóa học)	23
3.6.2. Bảng CourseProgress (Tổng hợp tiến độ khóa học)	24
3.6.3. Bảng LessonProgress (Chi tiết tiến độ từng bài học)	24
3.6.4. Bảng ProgressHistory (Nhật ký hoạt động học tập)	24
3.7. Thiết kế Redis Cache	25
3.8. Thiết kế Event Streaming (Kafka)	25
3.9. Thiết kế sơ đồ hệ thống (System Architecture Diagram)	26
3.10. Thiết kế sơ đồ tuần tự (Sequence Diagram)	28
3.10.1. Sequence Diagram: Đăng nhập	28
3.10.2. Sequence Diagram: Học viên ghi danh khóa học	28
3.10.3. Sequence Diagram: Hoàn thành bài học (CompleteLesson)	29
3.10.4. Sequence Diagram: Xem Dashboard tiến độ	29
3.11. Thiết kế sơ đồ triển khai (Deployment Diagram)	29
CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ	31
4.1. Công nghệ sử dụng	31
4.1.1. Giới thiệu về ASP.NET Core	31
4.1.2. Giới thiệu về React và TypeScript	31
4.1.3. Hệ quản trị cơ sở dữ liệu PostgreSQL	32
4.1.4. Hệ thống lưu trữ dữ liệu Redis	32
4.1.5. Công nghệ ảo hóa Docker và Docker Compose	33
4.1.6. Công nghệ ứng dụng	33
4.1.7. Đánh giá lựa chọn công nghệ	34
4.2. Demo hệ thống	35
4.2.1. Giao diện đăng nhập	35
4.2.2. Giao diện trang chủ	36
4.2.3. Giao diện khóa học	37
4.2.4. Giao diện thanh toán & ghi danh	38
4.2.5. Giao diện nội dung bài học và kiểm tra kiến thức với AI	39
4.2.6. Giao diện lộ trình học tập	40
4.2.7. Giao diện AI hỗ trợ	41
4.2.8. Giao diện thành tích & kỷ lục	42
4.2.9. Giao diện Profile	43
4.2.10. Giao diện thông báo	44
4.2.11. Giao diện trang chủ Admin	45
4.2.12. Giao diện trạng thái máy chủ Admin	46
4.2.13. Giao diện quản lý khóa học	47
4.2.14. Giao diện quản lý người dùng	48
4.2.15. Giao diện quản lý giảng viên	49
4.2.16. Giao diện quản lý của giảng viên	50

4.3. Kiểm thử hệ thống.....	50
4.4. Đánh giá hiệu năng.....	51
4.5. Ưu điểm và hạn chế.....	51
KẾT LUẬN.....	53
TÀI LIỆU THAM KHẢO.....	54

DANH MỤC HÌNH

Hình 3.1. Mô hình Microservices Architecture	15
Hình 3.2. Usecase tổng quát	19
Hình 3.3. Sơ đồ hệ thống	27
Hình 3.4. Sequence Diagram: Đăng nhập (Login qua Gateway → Auth)	28
Hình 3.5. Sequence Diagram: Học viên ghi danh khóa học	28
Hình 3.6. Sequence Diagram: Hoàn thành bài học (CompleteLesson)	29
Hình 3.7. Sequence Diagram: Xem Dashboard tiến độ	29
Hình 4.1. Giao diện đăng nhập	35
Hình 4.2. Giao diện trang chủ	36
Hình 4.3. Giao diện khóa học	37
Hình 4.4. Giao diện thanh toán & ghi danh	38
Hình 4.5. Giao diện học và kiểm tra kiến thức với AI	39
Hình 4.6. Giao diện lộ trình học tập	40
Hình 4.7. Giao diện AI hỗ trợ	41
Hình 4.8. Giao diện bảng vàng	42
Hình 4.9. Giao diện profile	43
Hình 4.10. Giao diện thông báo	44
Hình 4.11. Giao diện trang chủ Admin	45
Hình 4.12. Giao diện giám sát hạ tầng	46
Hình 4.13. Giao diện quản lý khóa học	47
Hình 4.14. Giao diện quản lý người dùng	48
Hình 4.15. Giao diện quản lý giảng viên	49
Hình 4.16. Giao diện quản lý của giảng viên	50

DANH MỤC BẢNG

Bảng 3.1. Usecase đăng ký tài khoản.....	20
Bảng 3.2. Usecase Xem Dashboard và Tiến độ học tập	21
Bảng 3.3. Usecase quản lý nội dung bài giảng	21
Bảng 3.4. Usecase Nhận tư vấn lộ trình học tập (AI Advisor)	22
Bảng 3.5. Usecase Quản lý tài khoản và Phân quyền.....	22
Bảng 4.1. Tổng hợp ưu và nhược điểm của các công nghệ áp dụng	34

DANH SÁCH CÁC KÝ TỰ, CHỮ VIẾT TẮT

Từ viết tắt	Giải thích
LMS	Learning Management System – Hệ thống quản lý học tập trực tuyến.
API	Application Programming Interface – Giao diện lập trình ứng dụng dùng để kết nối các dịch vụ.
JWT	JSON Web Token – Một tiêu chuẩn mở dùng để truyền tải thông tin an toàn giữa các bên dưới dạng đối tượng JSON, thường dùng cho xác thực.
REST	Representational State Transfer – Một kiểu kiến trúc phần mềm cho các hệ thống phân tán, thường dùng HTTP làm giao thức truyền tải.
SPA	Single Page Application – Ứng dụng web một trang, giúp trải nghiệm người dùng mượt mà bằng cách không cần tải lại toàn bộ trang khi chuyển hướng.
RBAC	Role-Based Access Control – Kiểm soát truy cập dựa trên vai trò, dùng để phân quyền cho Học viên, Giảng viên và Admin.
DDD	Domain-Driven Design – Thiết kế hướng tên miền, giúp tổ chức mã nguồn microservices theo nghiệp vụ cốt lõi.
SPOF	Single Point of Failure – Điểm yếu tập trung, nơi nếu xảy ra sự cố sẽ khiến toàn bộ hệ thống ngừng hoạt động (như API Gateway).
BCrypt	Thuật toán băm mật khẩu bảo mật, giúp bảo vệ thông tin đăng nhập của người dùng.
TTL	Time To Live – Thời gian sống của dữ liệu trong bộ nhớ đệm Redis trước khi tự động bị xóa.
BI	Business Intelligence – Trí tuệ doanh nghiệp, trong dự án này dùng để phân tích dữ liệu học tập thông minh.
CORS	Cross-Origin Resource Sharing – Cơ chế cho phép các tài nguyên trên web được truy vấn từ một tên miền khác.
UML	Unified Modeling Language – Ngôn ngữ mô hình hóa thống nhất, dùng để vẽ các sơ đồ Use Case và sơ đồ tuần tự.

CI/CD	Continuous Integration / Continuous Deployment – Quy trình tích hợp và triển khai phần mềm tự động.
--------------	---

MỞ ĐẦU

Trong kỷ nguyên giáo dục 4.0, các hệ thống quản lý học tập (LMS) truyền thống với kiến trúc nguyên khối (Monolithic) đang dần bộc lộ những hạn chế về khả năng mở rộng và tốc độ xử lý dữ liệu thời gian thực. Việc người dùng phải đối mặt với tình trạng hệ thống chậm chạp hoặc thông tin tiến độ học tập không được cập nhật chính xác là những rào cản lớn trong trải nghiệm giáo dục trực tuyến.

Nhận thấy tầm quan trọng của việc tối ưu hóa hiệu suất và cá nhân hóa lộ trình học tập, tôi đã tập trung nghiên cứu và triển khai dự án IntelligentLMS. Điểm khác biệt cốt lõi của đề tài này so với các phần mềm quản lý thông thường chính là việc ứng dụng kiến trúc Microservices kết hợp cùng công nghệ Docker. Việc chia nhỏ hệ thống thành các dịch vụ độc lập như xác thực người dùng (Auth Service), quản lý khóa học (Course Service) và theo dõi tiến độ (Progress Service) không chỉ giúp hệ thống vận hành linh hoạt mà còn đảm bảo dữ liệu luôn được đồng bộ hóa tức thời. Học viên có thể theo dõi chính xác từng phần trăm tiến độ hoàn thành dựa trên dữ liệu thực tế được trích xuất trực tiếp từ hệ quản trị PostgreSQL. Đồng thời, hệ thống tích hợp thêm Redis đóng vai trò là lớp bộ nhớ đệm (Caching) trung gian, giúp truy xuất và hiển thị thông tin tiến độ lên Dashboard một cách tức thời, giảm thiểu độ trễ và tối ưu hóa hiệu suất truy vấn cho toàn hệ thống.

Kết hợp giữa nền tảng kiến thức về phát triển phần mềm hiện đại và kiến trúc hệ thống phân tán, tôi thực hiện đề tài “Xây dựng hệ thống quản lý học tập thông minh dựa trên kiến trúc Microservices - IntelligentLMS”. Đề tài hướng tới việc giải quyết bài toán hiệu năng của các hệ thống LMS hiện nay, đồng thời tích hợp các tính năng thông minh nhằm gợi ý lộ trình học tập chuyên sâu, giúp nâng cao hiệu quả giáo dục và chuyển đổi số toàn diện trong công tác quản lý đào tạo.

1. Lý do chọn đề tài

Trong bối cảnh cuộc Cách mạng Công nghiệp 4.0 đang tái định nghĩa lại toàn bộ các giá trị kinh tế và xã hội, giáo dục trực tuyến (E-learning) đã không còn đơn thuần là một giải pháp tình thế mà đã vươn mình trở thành một phương thức truyền tải tri thức chiến lược, tất yếu trong kỷ nguyên số. Sự bùng nổ của Internet và các thiết bị di động đã tạo

nên một làn sóng chuyển đổi số mạnh mẽ, giúp cá nhân hóa lộ trình học tập, tối ưu hóa nguồn lực thời gian và phá vỡ mọi rào cản về khoảng cách địa lý cho người học trên toàn thế giới. Tuy nhiên, thực trạng vận hành của các hệ thống quản lý học tập (LMS) truyền thống hiện nay đang bộc lộ những hạn chế mang tính hệ thống, đặt ra những thách thức nan giải cho cả người dạy lẫn người học.

Thứ nhất, về mặt hạ tầng kỹ thuật, đa số các nền tảng giáo dục hiện tại vẫn vận hành dựa trên kiến trúc đơn khối (Monolithic), một mô hình mà ở đó mọi chức năng được gắn chặt vào một khối duy nhất. Điều này dẫn đến khả năng chịu tải kém; khi số lượng người dùng tăng đột biến trong các giờ cao điểm, hệ thống thường xuyên rơi vào tình trạng phản hồi chậm hoặc ngưng trệ, gây gián đoạn nghiêm trọng đến tiến trình tiếp thu tri thức. Thứ hai, về mặt trải nghiệm người dùng, một bài toán khó đối với các hệ thống truyền thống là sự thiếu hụt trầm trọng trong việc trực quan hóa dữ liệu học tập. Người học thường cảm thấy đơn điệu, thiếu động lực khi phải đối mặt với các kho bài giảng khô khan mà không có một cơ chế theo dõi tiến độ cụ thể, sinh động để nắm bắt được vị trí của bản thân trên lộ trình chinh phục kiến thức.

Đặc biệt, trong một xã hội nơi trí tuệ nhân tạo (AI) đang trở thành "bộ phóng" cho mọi quyết định thông minh, giáo dục trực tuyến không thể đứng ngoài cuộc đua này. Việc thiếu vắng các công cụ tư vấn tự động, có khả năng thấu hiểu hành vi và đưa ra các gợi ý lộ trình cá nhân hóa đã khiến người học dễ dàng mất phương hướng giữa biển thông tin khổng lồ. Nhận thấy tầm quan trọng của việc giải quyết triệt để các rào cản trên, chúng tôi quyết định thực hiện đề tài “Xây dựng web quản lý học tập trực tuyến” với tên gọi IntelligentLMS.

2. Mục đích nghiên cứu

Mục đích nghiên cứu của đề tài là xây dựng hoàn thiện một hệ thống quản lý học tập trực tuyến có khả năng vận hành linh hoạt, an toàn và thông minh với các mục tiêu cụ thể sau:

- Về mặt kiến trúc: Nghiên cứu và hiện thực hóa mô hình Microservices, chia tách hệ thống thành các dịch vụ độc lập như xác thực người dùng (Auth Service), quản lý khóa học (Course Service) và theo dõi tiến độ (Progress Service).

- Về mặt chức năng: Xây dựng hệ thống theo dõi tiến độ học tập thời gian thực (Real-time Progress Tracking), cho phép đồng bộ hóa dữ liệu bài học một cách chính xác tuyệt đối giữa phía máy chủ và giao diện người dùng.

Về mặt công nghệ AI: Nghiên cứu cách thức tích hợp các mô hình ngôn ngữ lớn hoặc các thuật toán phân tích dữ liệu để xây dựng tính năng "Cố vấn thông minh", giúp đưa ra các lời khuyên học tập dựa trên dữ liệu hành vi của từng cá nhân.

3. Đối tượng nghiên cứu

Chúng tôi xác định các đối tượng nghiên cứu trọng tâm bao gồm cả khía cạnh người dùng, hạ tầng kỹ thuật và các công nghệ cốt lõi:

- Đối tượng người dùng và quản trị:
 - Người học (Students): Nghiên cứu các nhu cầu về việc theo dõi lộ trình học tập, tương tác với bài giảng và nhận phản hồi từ hệ thống.
 - Giảng viên và Quản trị viên (Lecturers & Admins): Nghiên cứu quy trình quản lý nội dung khóa học, giám sát tiến độ của học viên và điều phối tài nguyên giáo dục trực tuyến.
- Kiến trúc và Hạ tầng kỹ thuật:
 - Kiến trúc phân tán Microservices: Nghiên cứu cách thức chia tách và vận hành các dịch vụ độc lập như Auth, Course, User và Progress Service.
 - Cơ chế điều phối (API Gateway): Nghiên cứu vai trò của Gateway (Ocelot/YARP) trong việc tiếp nhận, bảo mật và điều hướng mọi yêu cầu từ phía người dùng đến các dịch vụ đích.
 - Cơ sở dữ liệu chuyên biệt: Nghiên cứu cách thức tổ chức dữ liệu quan hệ trên PostgreSQL, đảm bảo tính toàn vẹn và độc lập về dữ liệu cho từng Microservice.
- Cơ chế giao tiếp và xử lý dữ liệu:

- Giao tiếp bất đồng bộ (Message Broker): Nghiên cứu cơ chế truyền tin nhắn thông qua Apache Kafka để đảm bảo sự đồng bộ dữ liệu giữa các dịch vụ mà không gây tắc nghẽn hệ thống.
- Bảo mật và Xác thực: Nghiên cứu hệ thống Identity và JWT (JSON Web Token) để đảm bảo an toàn thông tin và phân quyền chính xác cho từng loại đối tượng người dùng.
- Trải nghiệm người dùng và Công nghệ thông minh:
 - Giao diện Dashboard tương tác: Nghiên cứu việc ứng dụng React kết hợp cùng các thư viện đồ họa (Framer Motion) để tạo ra các hiệu ứng chuyển động trực quan, giúp hiển thị tiến độ học tập một cách sinh động và hấp dẫn.
 - Module AI Advisor: Nghiên cứu việc tích hợp trí tuệ nhân tạo để phân tích dữ liệu tiến trình học tập, từ đó cung cấp các gợi ý lộ trình và lời khuyên cá nhân hóa (Personalized Recommendations).

Cơ chế Containerization: Nghiên cứu việc sử dụng Docker và Docker Compose để chuẩn hóa quy trình triển khai, giúp hệ thống có khả năng vận hành đồng nhất trên mọi môi trường.

4. Phương pháp nghiên cứu

Để đảm bảo tính khoa học và thực tiễn, chúng tôi đã áp dụng các phương pháp nghiên cứu sau:

- Phương pháp phân tích và thiết kế: Sử dụng các công cụ mô hình hóa để thiết kế sơ đồ kiến trúc hệ thống, sơ đồ luồng dữ liệu và thiết kế cơ sở dữ liệu chi tiết cho từng Microservice.
- Phương pháp thực nghiệm phần mềm: Áp dụng quy trình phát triển phần mềm hiện đại, lập trình Backend bằng ngôn ngữ C# trên nền tảng .NET 8 và Frontend bằng thư viện React.

- Phương pháp triển khai Containerization: Sử dụng Docker và Docker Compose để đóng gói toàn bộ môi trường ứng dụng, giúp đảm bảo tính đồng nhất của hệ thống từ môi trường phát triển đến môi trường triển khai thực tế.

Phương pháp đánh giá và kiểm thử: Thực hiện kiểm thử tích hợp (Integration Testing) để xử lý các lỗi phát sinh trong môi trường phân tán như lỗi CORS policy, lỗi đồng bộ dữ liệu và độ trễ mạng.

5. Phạm vi nghiên cứu

Phạm vi nghiên cứu của đề tài IntelligentLMS được giới hạn trong việc thiết kế và thực hiện các thành phần cốt lõi của một hệ sinh thái học tập trực tuyến hiện đại. Về khía cạnh chức năng, hệ thống tập trung vào việc xác thực người dùng, quản lý nội dung khóa học và xây dựng hệ thống theo dõi tiến độ thời gian thực thông qua dashboard tương tác cho ba nhóm đối tượng chính bao gồm người học, giảng viên và quản trị viên.

Về mặt kỹ thuật, đề tài ứng dụng kiến trúc Microservices triển khai trên nền tảng .NET 8 cho Backend và thư viện React cho Frontend, kết hợp cùng cơ chế điều phối API Gateway và hệ thống thông điệp bất đồng bộ Apache Kafka. Nghiên cứu cũng bao gồm việc tích hợp module AI Advisor để phân tích dữ liệu hành vi và cung cấp các gợi ý lộ trình học tập cá nhân hóa dựa trên dữ liệu nội bộ của hệ thống.

Để đảm bảo tính khả thi trong quá trình thực hiện, hệ thống được đóng gói và vận hành thông qua công nghệ Docker và Docker Compose. Phạm vi nghiên cứu này tạm thời chưa bao gồm việc xây dựng các cổng thanh toán trực tuyến hoặc thiết lập hạ tầng lưu trữ và phân phối video quy mô lớn để tập trung tối ưu hóa hiệu suất của kiến trúc phân tán và độ chính xác của các thuật toán gợi ý.

CHƯƠNG 1. TỔNG QUAN VỀ HỆ THỐNG LMS

1.1. GIỚI THIỆU VỀ LEARNING MANAGEMENT SYSTEM (LMS)

Learning Management System (LMS) được xem là một trong những nền tảng công nghệ trọng yếu trong quá trình chuyển đổi số giáo dục. LMS là hệ thống phần mềm được thiết kế nhằm hỗ trợ quản lý, tổ chức, triển khai và giám sát các hoạt động đào tạo trong môi trường trực tuyến. Khác với các hệ thống lưu trữ tài liệu thông thường, LMS không chỉ cung cấp chức năng phân phối nội dung mà còn tích hợp các công cụ quản lý người học, quản lý khóa học, tổ chức kiểm tra – đánh giá, theo dõi tiến độ và tổng hợp báo cáo thống kê [1].

Sự hình thành và phát triển của LMS gắn liền với nhu cầu mở rộng mô hình đào tạo ngoài phạm vi lớp học truyền thống. Trong giai đoạn đầu, LMS chủ yếu được sử dụng như một công cụ hỗ trợ học tập từ xa. Tuy nhiên, cùng với sự phát triển mạnh mẽ của hạ tầng mạng, điện toán đám mây và các công nghệ web hiện đại, LMS đã chuyển mình trở thành một hệ sinh thái đào tạo toàn diện. Hệ thống không chỉ đóng vai trò trung gian truyền tải nội dung mà còn là trung tâm điều phối toàn bộ hoạt động học tập, từ đăng ký khóa học, tương tác giữa giảng viên và người học, đến đánh giá và lưu trữ kết quả [2].

Trong bối cảnh giáo dục số, LMS góp phần thay đổi cách tiếp cận dạy và học. Thay vì phụ thuộc hoàn toàn vào không gian và thời gian cố định, người học có thể tiếp cận nội dung linh hoạt, chủ động quản lý tiến độ học tập của bản thân. Đồng thời, giảng viên có thể theo dõi mức độ tham gia, phân tích hành vi học tập và điều chỉnh phương pháp giảng dạy phù hợp hơn với từng nhóm đối tượng [3].

Thực tiễn triển khai cho thấy hiệu quả của LMS không chỉ phụ thuộc vào công nghệ mà còn chịu tác động mạnh mẽ bởi yếu tố sự phạm và trải nghiệm người dùng. Một hệ thống LMS được thiết kế tốt cần đảm bảo sự cân bằng giữa tính ổn định kỹ thuật, khả năng mở rộng, và tính thân thiện trong giao diện. Do đó, việc nghiên cứu và xây dựng LMS đòi hỏi sự kết hợp giữa kiến thức công nghệ thông tin và nguyên lý giáo dục, nhằm tạo ra môi trường học tập số hiệu quả và bền vững [4].

1.2. VAI TRÒ CỦA LMS TRONG GIÁO DỤC SỐ

Trong bối cảnh giáo dục đại học đang chịu tác động mạnh mẽ từ chuyển đổi số, Learning Management System (LMS) đóng vai trò như một hạ tầng công nghệ trung tâm trong quản lý và tổ chức hoạt động đào tạo. Không chỉ là công cụ hỗ trợ giảng dạy trực tuyến, LMS còn góp phần tái cấu trúc phương thức vận hành của cơ sở giáo dục theo hướng linh hoạt, minh bạch và hiệu quả hơn [5].

Thứ nhất, LMS hỗ trợ chuẩn hóa quy trình quản lý đào tạo. Thông qua hệ thống, toàn bộ thông tin về khóa học, danh sách sinh viên, nội dung bài giảng, lịch học, bài tập và kết quả đánh giá được lưu trữ tập trung và quản lý thống nhất. Điều này giúp giảm thiểu sai sót trong quá trình vận hành, đồng thời nâng cao tính minh bạch trong quản lý học vụ [6].

Thứ hai, LMS thúc đẩy mô hình học tập linh hoạt (flexible learning). Sinh viên có thể truy cập tài liệu, nộp bài và tham gia thảo luận ở bất kỳ thời điểm nào, không phụ thuộc hoàn toàn vào lịch học cố định. Điều này đặc biệt quan trọng trong các chương trình đào tạo kết hợp (blended learning) hoặc đào tạo từ xa [7].

Thứ ba, LMS tạo điều kiện tăng cường tương tác giữa giảng viên và người học. Thông qua các công cụ như diễn đàn thảo luận, hệ thống nhắn tin nội bộ, bài kiểm tra trực tuyến và phản hồi tự động, mức độ gắn kết trong môi trường học tập số được cải thiện đáng kể. Tương tác này không chỉ dừng lại ở việc trao đổi thông tin mà còn hỗ trợ hình thành cộng đồng học tập [8].

Bên cạnh đó, LMS còn hỗ trợ phân tích dữ liệu học tập. Dựa trên dữ liệu về tần suất truy cập, thời gian học tập, kết quả bài kiểm tra và mức độ hoàn thành khóa học, nhà trường và giảng viên có thể đánh giá xu hướng học tập của sinh viên, từ đó đưa ra các quyết định điều chỉnh phù hợp.

1.3. CÁC MÔ HÌNH KIẾN TRÚC PHỔ BIẾN

Trong quá trình xây dựng hệ thống phần mềm, việc lựa chọn mô hình kiến trúc đóng vai trò quyết định đến khả năng mở rộng, bảo trì và hiệu năng của hệ thống. Đối

với các nền tảng LMS nói riêng và hệ thống web nói chung, hai mô hình kiến trúc phổ biến hiện nay là Monolithic Architecture và Microservices Architecture. Mỗi mô hình có đặc điểm, ưu điểm và hạn chế riêng, phù hợp với từng quy mô và mục tiêu phát triển khác nhau.

1.3.1. Monolithic Architecture

Monolithic Architecture là mô hình kiến trúc truyền thống, trong đó toàn bộ hệ thống được xây dựng và triển khai như một khối thống nhất. Tất cả các thành phần như giao diện người dùng, xử lý nghiệp vụ và truy xuất dữ liệu được tích hợp trong cùng một ứng dụng và chia sẻ chung một cơ sở dữ liệu [9].

Trong mô hình này, các chức năng của hệ thống được tổ chức thành các module nội bộ nhưng vẫn thuộc cùng một mã nguồn và được triển khai đồng thời. Khi có thay đổi ở một thành phần bất kỳ, toàn bộ hệ thống thường phải được biên dịch và triển khai lại [10].

Ưu điểm của kiến trúc Monolithic là tính đơn giản trong giai đoạn đầu phát triển. Việc xây dựng, kiểm thử và triển khai tương đối dễ dàng do toàn bộ hệ thống nằm trong một cấu trúc thống nhất. Điều này phù hợp với các dự án có quy mô nhỏ hoặc đội ngũ phát triển hạn chế.

Tuy nhiên, khi hệ thống phát triển lớn hơn, kiến trúc Monolithic bộc lộ nhiều hạn chế. Việc mở rộng quy mô trở nên khó khăn do toàn bộ ứng dụng phải được mở rộng đồng bộ. Ngoài ra, sự phụ thuộc chặt chẽ giữa các module làm tăng rủi ro khi thay đổi hoặc nâng cấp hệ thống. Nếu một thành phần gặp lỗi nghiêm trọng, toàn bộ hệ thống có thể bị ảnh hưởng.

Đối với các hệ thống LMS quy mô nhỏ hoặc trong giai đoạn thử nghiệm, Monolithic Architecture có thể là lựa chọn phù hợp nhờ tính đơn giản và chi phí triển khai thấp.

1.3.2. Microservices Architecture

Microservices Architecture là mô hình kiến trúc hiện đại, trong đó hệ thống được chia thành nhiều dịch vụ nhỏ, độc lập và có thể triển khai riêng biệt. Mỗi dịch vụ đảm nhiệm một chức năng cụ thể, chẳng hạn như quản lý người dùng, quản lý khóa học, xử lý thanh toán hoặc thống kê dữ liệu [11].

Các dịch vụ trong mô hình Microservices giao tiếp với nhau thông qua các giao thức mạng, phổ biến nhất là HTTP/REST hoặc cơ chế truyền thông điệp. Mỗi dịch vụ có thể sử dụng cơ sở dữ liệu riêng và được phát triển, triển khai độc lập với các dịch vụ khác.

Ưu điểm nổi bật của kiến trúc Microservices là khả năng mở rộng linh hoạt. Khi một chức năng có lượng truy cập lớn, chỉ cần mở rộng dịch vụ tương ứng mà không ảnh hưởng đến toàn bộ hệ thống. Ngoài ra, mô hình này giúp tăng tính linh hoạt trong công nghệ, cho phép các dịch vụ sử dụng ngôn ngữ lập trình hoặc công nghệ khác nhau tùy theo yêu cầu [12].

Tuy nhiên, Microservices cũng đặt ra nhiều thách thức trong quản lý và vận hành. Việc phân tán hệ thống làm tăng độ phức tạp trong cấu hình, giám sát và bảo mật. Đồng thời, yêu cầu về hạ tầng triển khai và năng lực kỹ thuật của đội ngũ phát triển cũng cao hơn so với mô hình Monolithic [13].

Đối với các hệ thống LMS quy mô lớn, phục vụ nhiều người dùng và yêu cầu khả năng mở rộng cao, Microservices Architecture thường được xem là hướng tiếp cận phù hợp. Mô hình này hỗ trợ xây dựng hệ thống linh hoạt, dễ mở rộng và thích ứng với nhu cầu thay đổi trong tương lai [14].

1.4. SO SÁNH CÁC MÔ HÌNH KIẾN TRÚC

Việc đánh giá và lựa chọn mô hình kiến trúc đóng vai trò then chốt trong việc định hình khả năng vận hành của hệ thống quản lý học tập. Kiến trúc Monolithic và Microservices đại diện cho hai triết lý thiết kế đối lập, mỗi mô hình đều sở hữu những đặc tính kỹ thuật và giới hạn riêng biệt cần được xem xét kỹ lưỡng.

Trong mô hình Monolithic truyền thống, toàn bộ các thành phần chức năng từ giao diện người dùng, xử lý nghiệp vụ đến truy cập dữ liệu được đóng gói chặt chẽ trong một khối mã nguồn duy nhất. Cách tiếp cận này giúp đơn giản hóa quá trình phát triển ban đầu và kiểm thử cục bộ, tuy nhiên lại tạo ra rào cản lớn khi hệ thống cần mở rộng. Do sử dụng chung một cơ sở dữ liệu tập trung, kiến trúc đơn khối dễ rơi vào tình trạng nghẽn cổ chai dữ liệu và gặp khó khăn trong việc cô lập lỗi, nơi một sự cố nhỏ tại module xác thực có thể gây đình trệ toàn bộ hoạt động của ứng dụng [15].

Ngược lại, kiến trúc Microservices mà dự án IntelligentLMS áp dụng thực hiện phân rã hệ thống thành các dịch vụ độc lập như Auth, Course và Progress. Mỗi dịch vụ này hoạt động như một thực thể riêng biệt, sở hữu cơ sở dữ liệu độc lập và giao tiếp với nhau qua các giao thức mạng. Đặc tính này cho phép hệ thống đạt được khả năng mở rộng linh hoạt theo chiều ngang, nghĩa là nhóm nghiên cứu có thể tập trung phân bổ tài nguyên cho những dịch vụ có tải trọng lớn mà không cần nâng cấp toàn bộ hạ tầng [16].

Về khía cạnh triển khai và bảo trì, kiến trúc Microservices cho phép áp dụng quy trình CI/CD hiệu quả hơn thông qua công nghệ containerization của Docker. Trong khi Monolithic đòi hỏi phải đóng gói lại toàn bộ ứng dụng cho mỗi lần cập nhật nhỏ, Microservices cho phép nâng cấp và triển khai từng dịch vụ riêng lẻ một cách nhanh chóng. Mặc dù mô hình phân tán này đòi hỏi sự phức tạp trong việc điều phối thông điệp qua Apache Kafka và quản lý bộ nhớ đệm Redis, nhưng nó mang lại sự ổn định và hiệu năng vượt trội cho một nền tảng học tập đòi hỏi tính tương tác thời gian thực cao như IntelligentLMS [17].

1.5. ĐỊNH HƯỚNG XÂY DỰNG INTELLIGENT LMS

Dự án IntelligentLMS được định hướng phát triển không chỉ dừng lại ở một nền tảng quản lý học tập thông thường mà còn hướng tới một hệ sinh thái giáo dục thông minh và cá nhân hóa trải nghiệm người dùng. Định hướng cốt lõi của dự án tập trung vào việc chuyển đổi phương thức tiếp nhận kiến thức từ bị động sang chủ động thông qua các nền tảng công nghệ hiện đại.

Trụ cột đầu tiên trong định hướng xây dựng là việc hiện thực hóa mô hình cá nhân hóa lộ trình học tập thông qua module AI Advisor. Bằng cách phân tích dữ liệu hành vi và tiến độ học tập thực tế được ghi nhận liên tục trong PostgreSQL và Redis, hệ thống sẽ tự động đưa ra các gợi ý bài học, tài liệu bổ trợ hoặc bài kiểm tra phù hợp nhất với năng lực và tốc độ tiếp thu của từng học viên [18].

Trụ cột thứ hai tập trung vào trải nghiệm tương tác thời gian thực và tính trực quan hóa dữ liệu. Hệ thống hướng tới việc cung cấp các Dashboard thông minh, nơi mọi thông tin về tiến trình học tập (ví dụ: con số hoàn thành bài học đạt 30%) được tính toán và hiển thị tức thời nhờ sự phối hợp nhịp nhàng giữa Frontend React và dịch vụ Progress Service phía Backend [19].

Trụ cột cuối cùng là khả năng mở rộng bền vững và tính sẵn sàng cao dựa trên hạ tầng Docker và cụm Kafka Cluster. Định hướng này đảm bảo IntelligentLMS luôn có thể tích hợp thêm các module thông minh mới hoặc mở rộng quy mô phục vụ hàng ngàn học viên cùng lúc mà không làm ảnh hưởng đến cấu trúc nền tảng hiện tại. Tổng hòa các yếu tố này giúp hệ thống trở thành một người trợ lý học tập đặc lực, tối ưu hóa quy trình đào tạo trong môi trường số hóa.

CHƯƠNG 2. PHÂN TÍCH YÊU CẦU HỆ THỐNG

2.1. MÔ TẢ BÀI TOÁN

Sự bùng nổ của giáo dục trực tuyến đã đặt ra những thách thức lớn về mặt hạ tầng công nghệ cho các hệ thống quản lý học tập (LMS). Phần lớn các nền tảng hiện nay vẫn dựa trên kiến trúc Monolithic, nơi mọi logic nghiệp vụ được đóng gói trong một khối duy nhất, dẫn đến hiện tượng nghẽn cổ chai khi số lượng người dùng truy cập cùng lúc tăng cao. Bên cạnh đó, việc thiếu hụt khả năng cá nhân hóa khiến lộ trình học tập trở nên rập khuôn, không phản ánh đúng năng lực và tốc độ tiếp thu của từng cá nhân [20].

Hệ thống IntelligentLMS được xây dựng nhằm giải quyết triệt để các hạn chế này thông qua việc triển khai kiến trúc Microservices phân tán trên nền tảng .NET 8. Bài toán cốt lõi là thiết kế một hệ sinh thái có khả năng tự động hóa việc theo dõi tiến độ học tập theo thời gian thực và cung cấp các dashboard trực quan giúp học viên nắm bắt trạng thái hoàn thành bài học (ví dụ: mức đạt được 30%) ngay lập tức. Đồng thời, hệ thống cần tích hợp trí tuệ nhân tạo để phân tích hành vi và đưa ra các gợi ý lộ trình thông minh, chuyển đổi từ mô hình quản lý bị động sang hỗ trợ chủ động cho người học [21].

2.2. CÁC TÁC NHÂN HỆ THỐNG

Trong mô hình vận hành của IntelligentLMS, các tác nhân được phân định rạch ròi về quyền hạn và trách nhiệm để đảm bảo tính an toàn dữ liệu và tối ưu hóa quy trình nghiệp vụ.

Tác nhân Người học (Student) đóng vai trò là đối tượng trung tâm hưởng lợi từ hệ thống. Người học thực hiện các tương tác chính như đăng ký khóa học, truy cập nội dung bài giảng đa phương tiện, thực hiện các bài kiểm tra đánh giá và tương tác với module AI Advisor để nhận lộ trình cá nhân hóa. Toàn bộ hành vi của người học sẽ được hệ thống ghi nhận để hiển thị trên Dashboard tiến độ cá nhân [1].

Tác nhân Giảng viên (Lecturer) tập trung vào công tác quản lý chuyên môn và nội dung đào tạo. Giảng viên có quyền biên soạn giáo trình, cập nhật bài giảng video, quản lý ngân hàng câu hỏi và giám sát chi tiết quá trình học tập của các lớp học được phân công để đưa ra các can thiệp sư phạm kịp thời [17].

Tác nhân Quản trị viên (Admin) chịu trách nhiệm về tính sẵn sàng và ổn định của toàn bộ nền tảng. Các công việc chính bao gồm quản trị tài khoản người dùng, cấu hình phân quyền hệ thống (RBAC), giám sát trạng thái vận hành của các Microservices và tinh chỉnh các tham số cho mô hình AI Advisor để đảm bảo các gợi ý đưa ra là chính xác nhất [22].

2.3. CHỨC NĂNG HỆ THỐNG

Hệ thống cung cấp các chức năng xác thực và phân quyền người dùng, bao gồm đăng ký tài khoản, đăng nhập, đăng nhập bằng Google, sử dụng refresh token, quên mật khẩu thông qua OTP và đặt lại mật khẩu. Việc xác thực được thực hiện bằng JWT nhằm đảm bảo an toàn khi truy cập hệ thống. Ngoài ra, hệ thống còn hỗ trợ phân quyền theo vai trò như Admin, Teacher và User [23] [24].

Đối với quản lý người dùng, hệ thống cho phép người dùng xem thông tin cá nhân và cập nhật các thông tin như tên, avatar, số điện thoại và các thông tin liên quan khác.

Về quản lý khóa học, người dùng có thể xem danh sách khóa học, xem chi tiết từng khóa học, danh sách bài học trong khóa và số lượng bài học. Đối với giảng viên, hệ thống hỗ trợ xem và quản lý các khóa học do mình tạo [25].

Hệ thống cũng cung cấp chức năng ghi danh và theo dõi tiến độ học tập. Người dùng có thể ghi danh khóa học, kiểm tra trạng thái ghi danh, xem danh sách các khóa học đã đăng ký, theo dõi tiến độ học theo từng khóa và đánh dấu hoàn thành bài học [26].

Bên cạnh đó, hệ thống tích hợp chức năng thanh toán thông qua VNPAY, cho phép tạo link thanh toán, xử lý kết quả thanh toán, xác thực dữ liệu thanh toán bằng HMAC và tự động ghi danh vào khóa học sau khi thanh toán thành công [27].

Trong quá trình học tập, hệ thống hỗ trợ các chức năng như hiển thị dashboard bao gồm danh sách khóa học và tiến độ học, tiếp tục học các bài đang dở, xem nội dung bài học dưới dạng video hoặc tài liệu, đánh dấu hoàn thành bài học và tạo lộ trình học tập cá nhân.

Hệ thống còn cung cấp các chức năng về thành tích và thông báo, bao gồm hiển thị huy hiệu dựa trên quá trình học tập, hiển thị danh sách thông báo và hỗ trợ lọc thông báo.

Ngoài ra, hệ thống tích hợp AI để hỗ trợ người học, bao gồm gợi ý khóa học, tạo lộ trình học theo mục tiêu, phân tích tiến độ học và đưa ra cảnh báo nguy cơ bỏ học.

Đối với quản trị hệ thống, Admin có thể quản lý người dùng và khóa học (CRUD), cũng như lọc dữ liệu theo vai trò và trạng thái. Trong khi đó, Teacher có thể xem thống kê và quản lý các khóa học do mình phụ trách.

Cuối cùng, hệ thống được xây dựng với các thành phần hạ tầng hỗ trợ như API Gateway để định tuyến request, Redis để cache dữ liệu, Kafka để giao tiếp giữa các service và Docker để triển khai toàn bộ hệ thống [28].

2.4. YÊU CẦU PHI CHỨC NĂNG

Để vận hành hiệu quả trong kiến trúc phân tán, hệ thống đặt ra các tiêu chuẩn phi chức năng nghiêm ngặt về hiệu năng, bảo mật và khả năng mở rộng.

Tính sẵn sàng (Availability): Hệ thống phải đảm bảo hoạt động liên tục 24/7. Nhờ kiến trúc Microservices, nếu một dịch vụ như AI Advisor gặp sự cố, các chức năng cốt lõi như xem bài giảng vẫn phải hoạt động bình thường [29].

Hiệu năng và độ trễ (Performance): Tốc độ phản hồi Dashboard tiến độ phải dưới 1 giây. Điều này đạt được bằng cách sử dụng Redis làm lớp bộ nhớ đệm, giúp giảm thiểu các truy vấn nặng nề vào cơ sở dữ liệu PostgreSQL [30].

Khả năng mở rộng (Scalability): Hệ thống phải dễ dàng mở rộng theo chiều ngang bằng cách nhân bản các container Docker của các dịch vụ có tải trọng cao như Progress Service khi số lượng người dùng tăng đột biến [31].

Tính bảo mật (Security): Toàn bộ các yêu cầu từ Client phải đi qua API Gateway để thực hiện kiểm tra xác thực và ngăn chặn các cuộc tấn công phổ biến. Dữ liệu nhạy cảm như mật khẩu phải được mã hóa trước khi lưu trữ vào database [32].

2.5. PHÂN TÍCH RỦI RO HỆ THỐNG

2.5.1. Rủi ro về tính nhất quán dữ liệu (Eventual Consistency)

Trong kiến trúc Microservices hướng sự kiện, việc duy trì tính nhất quán tức thời (Strong Consistency) giữa các dịch vụ là cực kỳ khó khăn. Khi một học viên hoàn thành một bài học, Course Service sẽ phát đi một sự kiện thông qua Apache Kafka để Progress Service cập nhật tiến độ. Tuy nhiên, rủi ro về độ trễ mạng hoặc tình trạng quá tải của các Consumer có thể dẫn đến hiện tượng dữ liệu chưa được đồng bộ kịp thời. Hệ quả là học viên có thể thấy tiến độ trên Dashboard vẫn ở mức cũ (ví dụ 30%) dù đã hoàn thành thêm nội dung mới, gây ra sự nhầm lẫn về trải nghiệm người dùng. Để giảm thiểu rủi ro

này, hệ thống cần triển khai các cơ chế như Idempotent Consumer hoặc cơ chế bù đắp dữ liệu khi có sự cố giao tiếp giữa các Microservices [33] [34].

2.5.2. Rủi ro về vận hành và giám sát (Operational Complexity)

Việc quản lý đồng thời nhiều dịch vụ độc lập như Auth Service, Course Service và Progress Service trên nền tảng Docker làm tăng đáng kể khối lượng công việc quản trị hạ tầng. Mỗi dịch vụ có một vòng đời riêng, yêu cầu các bản cập nhật và cấu hình khác nhau, dẫn đến nguy cơ xung đột phiên bản hoặc lỗi cấu hình trong file Docker Compose. Hơn nữa, việc giám sát (Observability) trong môi trường phân tán trở nên thách thức hơn khi một yêu cầu từ người dùng đi qua nhiều lớp trung gian. Nếu không có hệ thống quản lý Log tập trung và Distributed Tracing, việc truy vết nguyên nhân gốc rễ của các lỗi logic sẽ tốn rất nhiều thời gian và nguồn lực [35] [36].

2.5.3. Rủi ro tại tầng API Gateway (Security & SPOF)

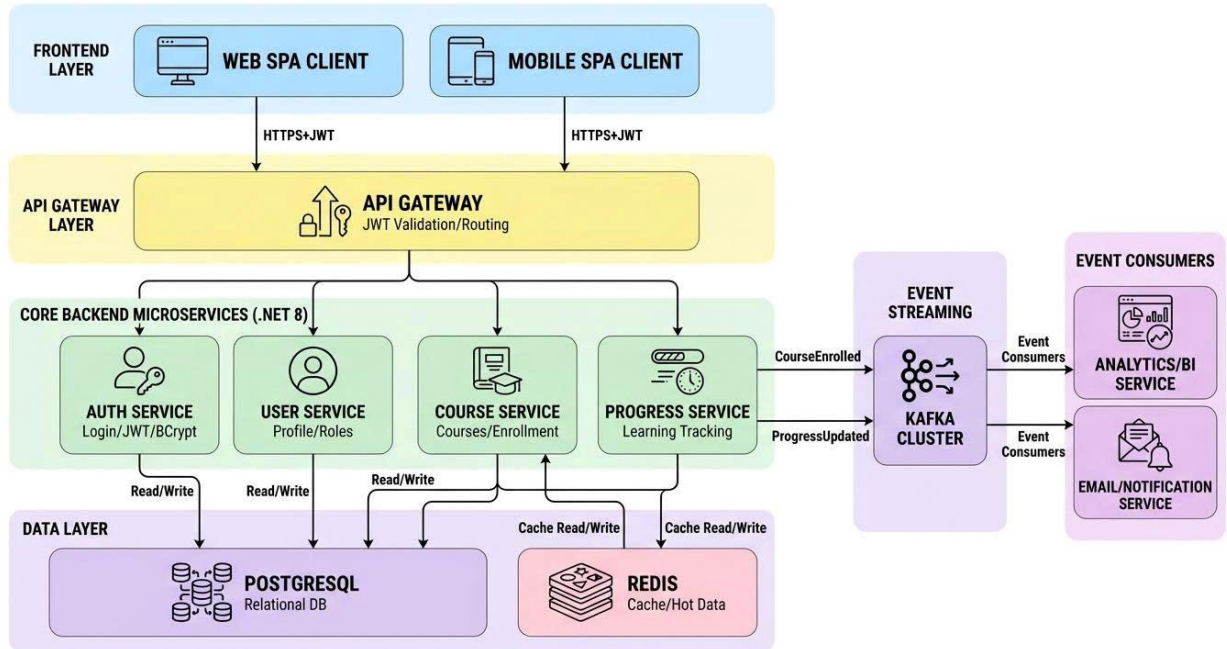
API Gateway đóng vai trò là cửa ngõ duy nhất vận hành tại cổng 5000, tiếp nhận và điều phối mọi yêu cầu từ phía Client. Điều này vô tình biến API Gateway trở thành điểm yếu tập trung của toàn hệ thống (Single Point of Failure). Nếu Gateway gặp sự cố hoặc bị tấn công từ chối dịch vụ (DDoS), toàn bộ ứng dụng IntelligentLMS sẽ bị tê liệt hoàn toàn mặc dù các dịch vụ Backend vẫn đang hoạt động tốt. Bên cạnh đó, các rủi ro về bảo mật tại tầng này như lộ lọt JWT hoặc cấu hình sai chính sách CORS có thể dẫn đến việc kẻ xấu truy cập trái phép vào các Microservices nội bộ để đánh cắp thông tin khóa học hoặc thay đổi tiến độ học tập của người dùng [37].

2.5.4. Rủi ro về độ chính xác của AI Advisor (AI Accuracy)

Hiệu quả của module AI Advisor phụ thuộc hoàn toàn vào chất lượng và khối lượng dữ liệu được lưu trữ trong PostgreSQL. Nếu tập dữ liệu hành vi của học viên quá ít hoặc bị nhiễu do các thao tác thử nghiệm, thuật toán AI sẽ đưa ra các gợi ý lộ trình không chính xác hoặc không liên quan đến nhu cầu thực tế. Rủi ro "khởi đầu lạnh" (Cold Start) khi hệ thống chưa có đủ lịch sử học tập cũng khiến AI Advisor gặp khó khăn trong việc đưa ra khuyến nghị giá trị ngay từ đầu. Điều này đòi hỏi quản trị viên phải liên tục giám sát, thực hiện kiểm thử A/B và tinh chỉnh thuật toán dựa trên phản hồi thực tế để đảm bảo tính "thông minh" của hệ thống [38].

CHƯƠNG 3. THIẾT KẾ HỆ THỐNG

3.1. KIẾN TRÚC TỔNG THỂ HỆ THỐNG



Hình 3.1. Mô hình Microservices Architecture

Đây là kiến trúc tổng thể của một hệ thống phần mềm được xây dựng theo mô hình Microservices kết hợp Event-Driven Architecture. Ở tầng trên cùng là Frontend Layer, bao gồm Web SPA Client và Mobile SPA Client. Hai client này giao tiếp với hệ thống thông qua giao thức HTTPS và sử dụng JWT để xác thực người dùng.

Bên dưới là API Gateway Layer, đóng vai trò là cổng trung gian tiếp nhận toàn bộ request từ client. API Gateway thực hiện kiểm tra JWT, xác thực bảo mật và định tuyến yêu cầu đến đúng microservice tương ứng trong hệ thống backend.

Tầng Core Backend Microservices (.NET 8) bao gồm nhiều dịch vụ độc lập như Auth Service (xử lý đăng nhập và phát hành JWT), User Service (quản lý thông tin và phân quyền người dùng), Course Service (quản lý khóa học và đăng ký học), và Progress Service (theo dõi tiến độ học tập). Các service này hoạt động tách biệt nhưng có thể phối hợp với nhau để hoàn thành nghiệp vụ.

Ở tầng Data Layer, hệ thống sử dụng PostgreSQL làm cơ sở dữ liệu quan hệ để lưu trữ dữ liệu chính và Redis làm bộ nhớ đệm (cache) nhằm tăng tốc độ truy xuất dữ liệu thường xuyên sử dụng.

Ngoài luồng xử lý đồng bộ thông qua API Gateway, hệ thống còn có cơ chế Event Streaming sử dụng Kafka Cluster. Khi xảy ra các sự kiện như đăng ký khóa học (CourseEnrolled) hoặc cập nhật tiến độ (ProgressUpdated), các service sẽ gửi sự kiện vào Kafka. Những dịch vụ khác như Analytics/BI Service hoặc Email/Notification Service sẽ lắng nghe và xử lý các sự kiện này theo cơ chế bất đồng bộ.

3.2. THIẾT KẾ API GATEWAY

Tầng API Gateway được thiết kế như một máy chủ proxy ngược (Reverse Proxy) đóng vai trò là cửa ngõ duy nhất tiếp nhận mọi yêu cầu từ các ứng dụng Web SPA và Mobile SPA. Việc tập trung hóa các tác vụ quản lý tại Gateway giúp giảm bớt gánh nặng xử lý cho các dịch vụ Backend và tăng cường tính bảo mật cho toàn bộ hạ tầng Microservices.

Các chức năng chi tiết của API Gateway bao gồm:

- Xác thực và phân quyền (Authentication & Authorization): Gateway thực hiện kiểm tra tính hợp lệ của mã báo JWT (JSON Web Token) đính kèm trong mỗi yêu cầu HTTPS. Chỉ những yêu cầu có chữ ký hợp lệ mới được phép đi sâu vào các dịch vụ nội bộ, giúp ngăn chặn các cuộc tấn công truy cập trái phép ngay từ lớp vỏ ngoài cùng.
- Điều hướng yêu cầu (Routing): Dựa trên URL của yêu cầu, Gateway sẽ thực hiện ánh xạ và chuyển tiếp yêu cầu đến đúng địa chỉ IP và cổng của microservice tương ứng trong cụm Docker. Ví dụ, các yêu cầu bắt đầu bằng /api/courses sẽ được điều hướng về Course Service.
- Cân bằng tải (Load Balancing): Gateway có khả năng phân phối lưu lượng truy cập đều cho nhiều phiên bản (instances) của cùng một dịch vụ đang chạy song song, giúp hệ thống duy trì hiệu suất ổn định khi lượng học viên truy cập Dashboard tăng đột biến.

- Giới hạn lưu lượng (Rate Limiting): Để bảo vệ hệ thống khỏi các cuộc tấn công từ chối dịch vụ hoặc spam API, Gateway được cấu hình để giới hạn số lượng yêu cầu tối đa từ một địa chỉ IP trong một khoảng thời gian nhất định..

3.3. THIẾT KẾ CÁC BACKEND SERVICES

3.3.1. Auth Service (Dịch vụ xác thực)

Đây là dịch vụ chịu trách nhiệm chính về an ninh tài khoản. Khi người dùng thực hiện đăng ký hoặc đăng nhập, Auth Service sẽ tiếp nhận thông tin và thực hiện băm mật khẩu bằng thuật toán BCrypt trước khi lưu trữ vào PostgreSQL. Sau khi xác thực thành công, dịch vụ sẽ phát hành một chuỗi JWT chứa các thông tin định danh và quyền hạn của người dùng để sử dụng cho các phiên làm việc tiếp theo.

3.3.2. User Service (Dịch vụ người dùng)

Dịch vụ này quản lý toàn bộ vòng đời của hồ sơ người dùng trong hệ thống. Nó thực hiện các nghiệp vụ như cập nhật thông tin cá nhân, quản lý ảnh đại diện và đặc biệt là điều phối quyền hạn dựa trên vai trò (Role-Based Access Control - RBAC). User Service cung cấp các API để Admin có thể giám sát danh sách người dùng và điều chỉnh quyền truy cập của Giảng viên hoặc Học viên khi cần thiết.

3.3.3. Course Service (Dịch vụ khóa học)

Course Service đóng vai trò là kho lưu trữ và điều phối nội dung giáo dục. Dịch vụ này quản lý các thực thể như Danh mục (Categories), Khóa học (Courses), và Bài giảng (Lessons). Nó tích hợp chặt chẽ với PostgreSQL để lưu trữ thông tin bền vững và phối hợp với Redis để cung cấp danh sách khóa học phổ biến một cách nhanh chóng. Khi có một học viên mới ghi danh, dịch vụ này sẽ phát đi sự kiện CourseEnrolled vào Kafka Cluster để các dịch vụ khác cùng cập nhật dữ liệu.

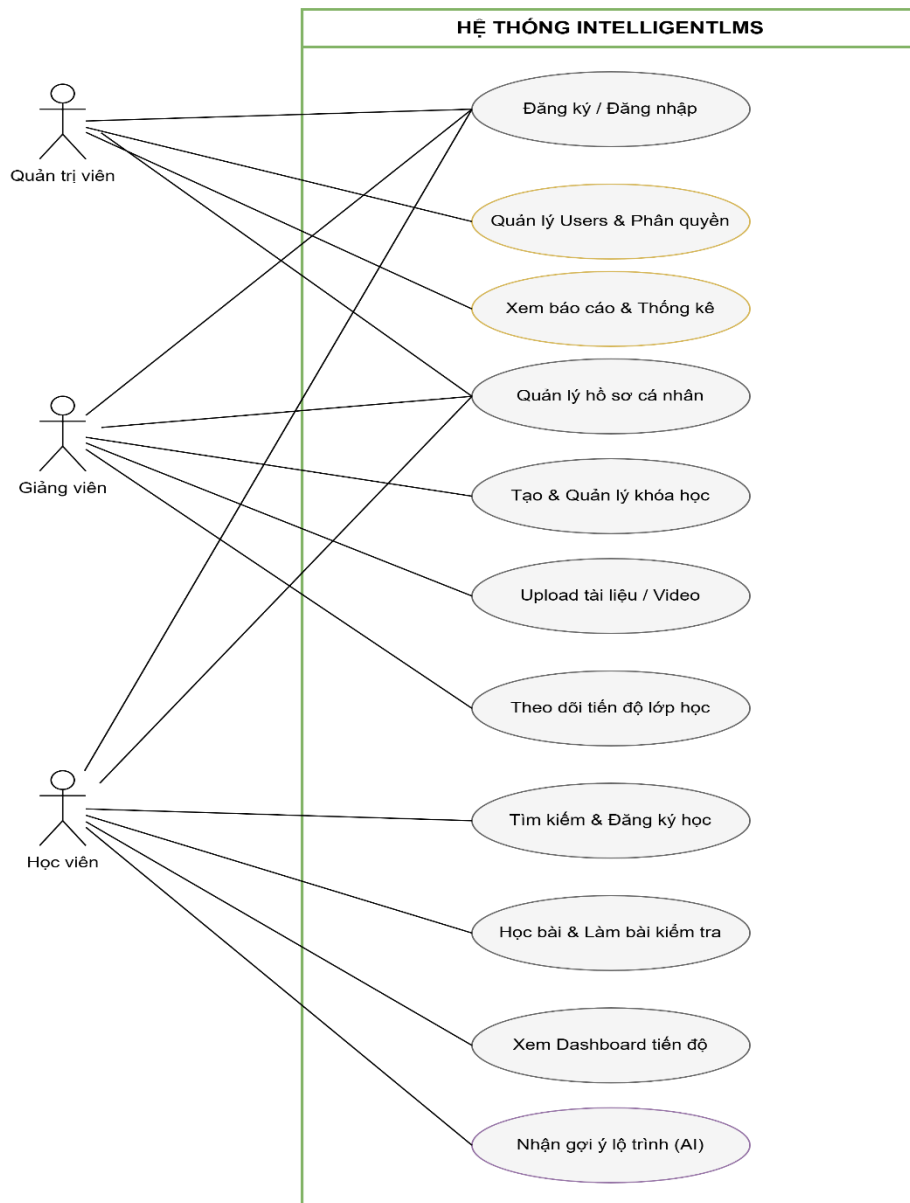
3.3.4. Progress Service (Dịch vụ tiến độ)

Đây là dịch vụ mang tính "thông minh" nhất của hệ thống, thực hiện nhiệm vụ Learning Tracking một cách chi tiết. Progress Service lắng nghe các hành động của học viên như hoàn thành video bài giảng hoặc nộp bài thi để tính toán lại tỷ lệ phần trăm tiến độ học tập.

Dịch vụ này sử dụng chiến lược lưu trữ kép:

- Lưu trữ nóng tại Redis: Để hiển thị tức thì các con số tiến độ (ví dụ 30%) lên Dashboard của học viên với độ trễ thấp nhất.
- Lưu trữ bền vững tại PostgreSQL: Để đảm bảo dữ liệu không bị mất mát và phục vụ cho các báo cáo thống kê dài hạn.
- Phát sự kiện ProgressUpdated: Mỗi khi tiến độ có sự thay đổi, một thông điệp sẽ được gửi qua Kafka để các dịch vụ Email hoặc Analytics có thể tiếp nhận và xử lý tiếp các bước như gửi thông báo chúc mừng hoặc phân tích hành vi học tập.

3.4. USE CASE DIAGRAM



Hình 3.2. Usecase tổng quát

Hệ thống được thiết kế phục vụ ba nhóm người dùng chính gồm: quản trị viên, giảng viên và sinh viên. Mỗi nhóm người dùng được phân quyền thực hiện các chức năng khác nhau nhưng đều phải thông qua cơ chế xác thực tài khoản trước khi truy cập hệ thống. Chức năng đăng ký và đăng nhập đóng vai trò là điểm khởi đầu, đảm bảo tính bảo mật và kiểm soát truy cập.

Đối với quản trị viên, hệ thống cung cấp các chức năng quản lý người dùng và phân quyền nhằm kiểm soát tài khoản, vai trò và quyền hạn của từng đối tượng sử dụng. Bên cạnh đó, quản trị viên có thể theo dõi báo cáo và thống kê tổng thể về tình

trạng hoạt động của hệ thống, bao gồm số lượng người dùng, khóa học và mức độ tương tác. Nhóm chức năng này hỗ trợ công tác quản lý và ra quyết định ở cấp hệ thống.

Giảng viên là tác nhân chịu trách nhiệm xây dựng và tổ chức nội dung đào tạo. Hệ thống cho phép giảng viên tạo mới và quản lý khóa học, cập nhật thông tin, tổ chức nội dung bài học và tải lên tài liệu hoặc video giảng dạy. Ngoài ra, giảng viên có thể theo dõi tiến độ học tập của sinh viên thông qua các công cụ thống kê và báo cáo nội bộ. Điều này giúp giảng viên đánh giá mức độ hoàn thành khóa học và kịp thời điều chỉnh phương pháp giảng dạy.

Đối với sinh viên, hệ thống hỗ trợ tìm kiếm và đăng ký khóa học phù hợp với nhu cầu cá nhân. Sau khi tham gia khóa học, sinh viên có thể truy cập nội dung bài học, thực hiện bài kiểm tra và theo dõi kết quả học tập. Dashboard tiến độ được cung cấp nhằm giúp sinh viên nắm bắt tổng quan quá trình học tập của bản thân, từ đó chủ động điều chỉnh kế hoạch học tập.

3.5. ĐẶC TẢ USECASE

3.5.1. Use Case: UC001 – Đăng ký tài khoản

Bảng 3.1. Usecase đăng ký tài khoản

Thành phần	Mô tả
Tên	Đăng ký tài khoản
Mô tả	Người dùng mới khởi tạo tài khoản để tham gia vào hệ thống IntelligentLMS.
Tác nhân	Người học (Student).
Điều kiện tiên quyết	Email hoặc tên đăng nhập không được trùng với dữ liệu đã tồn tại trong PostgreSQL.
Luồng chính	1. Người dùng chọn Đăng ký tại giao diện React. 2. Hệ thống hiển thị Form nhập liệu. 3. Người dùng nhập tên, email, mật khẩu. 4. Nhấn Đăng ký. 5. Auth Service xác thực và lưu thông tin vào database.

Luồng phụ	6. Nếu thông tin không hợp lệ hoặc email đã tồn tại, hệ thống hiển thị thông báo lỗi và yêu cầu nhập lại.
Điều kiện sau	Tài khoản được tạo thành công và người dùng có thể thực hiện đăng nhập để sử dụng hệ thống.

3.5.2. Use Case: UC002 – Xem Dashboard và Tiến độ học tập

Bảng 3.2. Usecase Xem Dashboard và Tiến độ học tập

Thành phần	Mô tả
Tên	Xem Dashboard và Tiến độ học tập
Mô tả	Người học theo dõi tổng quan các khóa học và phần trăm tiến độ hoàn thành bài học.
Tác nhân	Người học (Student).
Điều kiện tiên quyết	Người học đã đăng nhập thành công vào hệ thống.
Luồng chính	1. Sau khi đăng nhập, hệ thống mặc định hiển thị Dashboard. 2. Frontend React gọi API đến Progress Service thông qua Gateway. 3. Hệ thống truy xuất dữ liệu từ Redis (cache) hoặc PostgreSQL. 4. Dashboard hiển thị các biểu đồ tiến độ trực quan (ví dụ: hoàn thành 30%).
Luồng phụ	5. Nếu chưa có dữ liệu học tập, hệ thống hiển thị gợi ý bắt đầu khóa học mới.
Điều kiện sau	Người học nắm bắt được trạng thái học tập hiện tại của mình.

3.5.3. Use Case: UC003 – Quản lý nội dung bài giảng

Bảng 3.3. Usecase quản lý nội dung bài giảng

Thành phần	Mô tả
Tên	Biên soạn và Quản lý bài giảng
Mô tả	Giảng viên thực hiện việc cập nhật tài liệu, video và nội dung cho khóa học.
Tác nhân	Giảng viên (Lecturer).

Điều kiện tiên quyết	Giảng viên được cấp quyền quản lý khóa học cụ thể bởi Admin.
Luồng chính	1. Giảng viên chọn khóa học cần chỉnh sửa. 2. Chọn chức năng thêm mới bài giảng. 3. Upload tài liệu hoặc video lên hệ thống lưu trữ. 4. Cập nhật mô tả và nhấn Lưu. 5. Course Service ghi nhận thông tin và đồng bộ qua Kafka.
Luồng phụ	6. Nếu tệp tin upload vượt quá dung lượng cho phép, hệ thống yêu cầu kiểm tra lại định dạng tệp.
Điều kiện sau	Bài giảng mới được hiển thị cho các học viên đã ghi danh khóa học.

3.5.4. Use Case: UC004 – Nhận tư vấn lộ trình học tập (AI Advisor)

Bảng 3.4. Usecase Nhận tư vấn lộ trình học tập (AI Advisor)

Thành phần	Mô tả
Tên	Nhận tư vấn lộ trình thông minh
Mô tả	Hệ thống đưa ra gợi ý các bài học hoặc khóa học phù hợp dựa trên hành vi của người học.
Tác nhân	Người học (Student).
Điều kiện tiên quyết	Người học có lịch sử tương tác tối thiểu trong hệ thống.
Luồng chính	1. Người học truy cập vào module AI Advisor. 2. Hệ thống gọi AI Service để phân tích dữ liệu từ PostgreSQL. 3. Thuật toán xử lý và đưa ra các đề xuất cá nhân hóa. 4. Người học xem và chọn học theo lộ trình gợi ý.
Luồng phụ	5. Nếu dữ liệu quá ít, AI Advisor sẽ gợi ý các khóa học phổ biến của hệ thống.
Điều kiện sau	Người học có lộ trình học tập tối ưu hơn để cải thiện kết quả.

3.5.5. Use Case: UC005 – Quản lý tài khoản và Phân quyền

Bảng 3.5. Usecase Quản lý tài khoản và Phân quyền

Thành phần	Mô tả
------------	-------

Tên	Quản trị người dùng và Phân quyền
Mô tả	Quản lý danh sách tài khoản và điều chỉnh quyền hạn truy cập của các thành viên.
Tác nhân	Quản trị viên (Admin).
Điều kiện tiên quyết	Đăng nhập với quyền tài khoản Admin cao nhất.
Luồng chính	1. Truy cập vào trang Quản trị người dùng. 2. Tìm kiếm học viên hoặc giảng viên cần điều chỉnh. 3. Chọn vai trò mới (Role) hoặc cập nhật trạng thái tài khoản. 4. Nhấn Lưu thay đổi. 5. Hệ thống cập nhật bảng Users và ghi nhật ký hệ thống.
Luồng phụ	6. Nếu tài khoản đang bị khóa, Admin có thể thực hiện mở khóa để người dùng tiếp tục sử dụng.
Điều kiện sau	Quyền hạn của người dùng được cập nhật ngay lập tức cho phiên làm việc tiếp theo.

3.6. THIẾT KẾ CƠ SỞ DỮ LIỆU

3.6.1. Bảng Enrollments (Quản lý đăng ký khóa học)

Tên trường	Kiểu dữ liệu	Ý nghĩa	Lý do lựa chọn/Logic xử lý
Id	UUID	Khóa chính của bản ghi.	Đảm bảo tính duy nhất trên toàn hệ thống Microservices.
UserId	UUID	Mã định danh người dùng.	Liên kết với dịch vụ xác thực (Auth Service).
CourseId	UUID	Mã định danh khóa học.	Liên kết với dịch vụ quản lý nội dung (Course Service).
EnrolledAt	Timestamp	Thời gian đăng ký.	Dùng để thống kê thời điểm tham gia của học viên.
Status	Varchar	Trạng thái (Active/Inactive).	Kiểm soát quyền truy cập vào nội dung học tập.

3.6.2. Bảng CourseProgress (Tổng hợp tiến độ khóa học)

Tên trường	Kiểu dữ liệu	Ý nghĩa	Lý do lựa chọn/Logic xử lý
TotalLessons	Int	Tổng số bài học.	Ví dụ: 10 bài học trong một khóa.
CompletedLessons	Int	Số bài đã hoàn thành.	Ví dụ: 3 bài học đã xem xong.
ProgressPercentage	Int	Phần trăm tiến độ.	Tính toán theo công thức: $\text{Progress} = \frac{\text{Completed}}{\text{Total}} \times 100\%$.
UpdatedAt	Timestamp	Thời điểm cập nhật.	Dùng để đồng bộ dữ liệu sang Redis nhằm tăng tốc Dashboard.

3.6.3. Bảng LessonProgress (Chi tiết tiến độ từng bài học)

Tên trường	Kiểu dữ liệu	Ý nghĩa	Lý do lựa chọn/Logic xử lý
LessonId	UUID	Mã định danh bài học.	Xác định chính xác bài học đang được theo dõi.
IsCompleted	Boolean	Trạng thái hoàn thành.	Đánh dấu bài học đã đạt yêu cầu hay chưa.
WatchTimeSeconds	Int	Thời gian đã xem (giây).	Dữ liệu để phân tích mức độ chuyên cần của học viên.
CompletedAt	Timestamp	Thời điểm hoàn thành.	Ghi nhận mốc thời gian bài học được đánh dấu xong.

3.6.4. Bảng ProgressHistory (Nhật ký hoạt động học tập)

Tên trường	Kiểu dữ liệu	Ý nghĩa	Lý do lựa chọn/Logic xử lý
ActionType	Varchar	Loại hành động.	Các giá trị: 'Started', 'Completed', 'Paused'.
CreatedAt	Timestamp	Thời gian phát sinh.	Dữ liệu thô để vẽ biểu đồ hoạt động theo tuần trên React.

3.7. THIẾT KẾ REDIS CACHE

Trong kiến trúc IntelligentLMS, Redis được triển khai như một tầng lưu trữ dữ liệu nóng (Hot Data) nằm giữa các dịch vụ Backend và cơ sở dữ liệu quan hệ PostgreSQL. Mục tiêu chính của thiết kế này là tối ưu hóa tốc độ phản hồi và giảm thiểu tải trọng cho hệ quản trị cơ sở dữ liệu chính khi xử lý các truy vấn có tần suất lặp lại cao.

Hệ thống ứng dụng Redis vào hai kịch bản nghiệp vụ trọng yếu:

Chiến lược Cache-Aside cho Course Service: Các thông tin về danh sách khóa học phổ biến hoặc cấu trúc bài giảng thường xuyên được truy cập sẽ được lưu vào Redis. Khi người dùng tìm kiếm, hệ thống ưu tiên kiểm tra dữ liệu trong Cache trước khi truy vấn PostgreSQL, giúp giảm thời gian phản hồi trang danh mục xuống mức mili giây.

Lưu trữ trạng thái tiến độ tức thời: Dịch vụ Progress Service thực hiện ghi nhận các thay đổi tiến độ học tập (Learning Tracking) trực tiếp vào Redis. Điều này đảm bảo rằng các con số phần trăm hoàn thành bài học trên Dashboard của học viên luôn được cập nhật và hiển thị ngay lập tức mà không cần chờ đợi quá trình ghi dữ liệu bền vững vào PostgreSQL kết thúc.

3.8. THIẾT KẾ EVENT STREAMING (KAFKA)

Hệ thống áp dụng kiến trúc hướng sự kiện (Event-driven Architecture) thông qua cụm Kafka Cluster để đảm bảo tính nhất quán dữ liệu và sự độc lập giữa các Microservices. Kafka đóng vai trò là trục xương sống cho việc truyền tin bất đồng bộ, giúp hệ thống có thể mở rộng quy mô mà không làm tăng độ trễ xử lý trực tiếp.

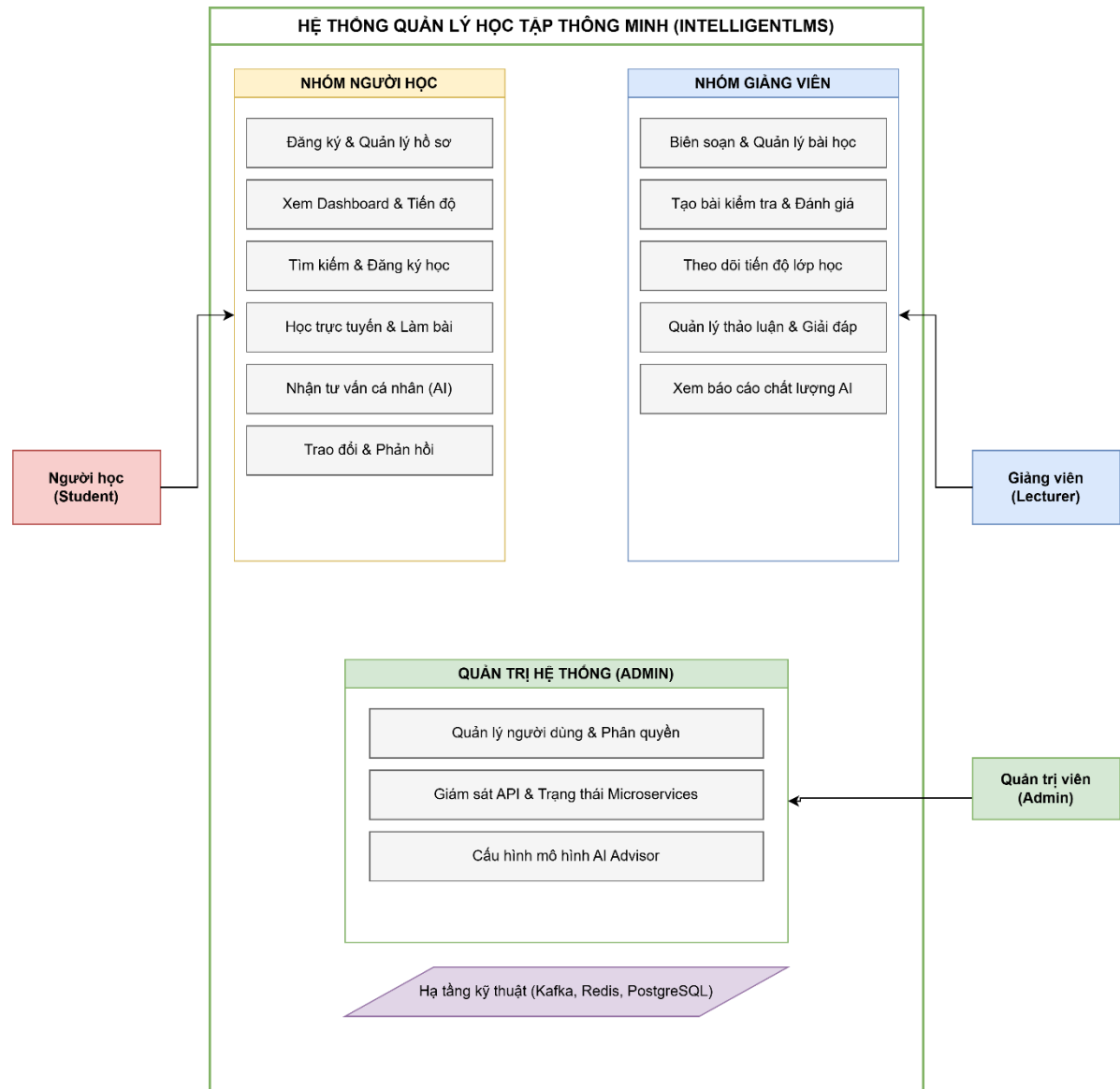
Cấu trúc dòng sự kiện trong IntelligentLMS bao gồm:

Phát hành sự kiện (Event Producers): Các dịch vụ cốt lõi như Course Service và Progress Service đóng vai trò là các nhà sản xuất thông điệp. Khi có một học viên đăng ký khóa học mới hoặc hoàn thành một mục tiêu học tập, các dịch vụ này sẽ đẩy các sự

kiện tương ứng như CourseEnrolled hoặc ProgressUpdated vào các Topic cụ thể trên Kafka.

Tiêu thụ sự kiện (Event Consumers): Các dịch vụ phụ trợ như Analytics/BI Service và Email/Notification Service sẽ đăng ký lắng nghe các Topic này. Khi có sự kiện mới, Analytics Service sẽ thực hiện phân tích hành vi học tập để AI Advisor đưa ra gợi ý, trong khi Email Service sẽ tự động gửi thông báo nhắc nhở hoặc chúc mừng cho học viên mà không gây ảnh hưởng đến hiệu suất của các dịch vụ nghiệp vụ chính.

3.9. THIẾT KẾ SƠ ĐỒ HỆ THỐNG (SYSTEM ARCHITECTURE DIAGRAM)



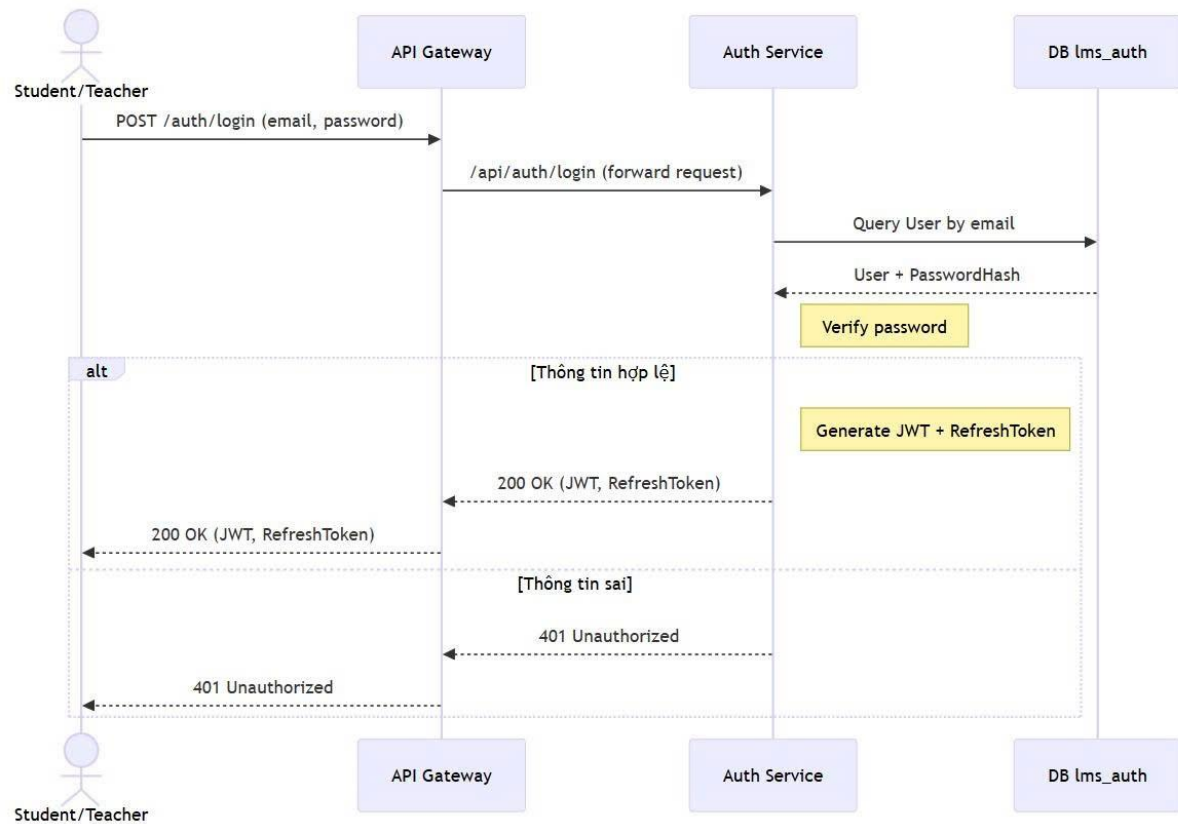
Hình 3.3. Sơ đồ hệ thống

- Nhóm Người học (Student): Đây là trung tâm của hệ thống, thực hiện các tương tác trực tiếp với nội dung giáo dục. Các chức năng dành cho nhóm này bao gồm: đăng ký và quản lý hồ sơ cá nhân; theo dõi tiến độ học tập trực quan trên Dashboard; tìm kiếm và ghi danh các khóa học mới; tham gia các bài giảng trực tuyến và thực hiện bài kiểm tra đánh giá. Đặc biệt, người học còn nhận được sự hỗ trợ từ chức năng tư vấn lộ trình cá nhân hóa dựa trên công nghệ AI (AI Advisor).
- Nhóm Giảng viên (Lecturer): Tập trung vào công tác chuyên môn và quản lý chất lượng đào tạo. Giảng viên có quyền biên soạn và quản lý bài học; xây dựng ngân hàng câu hỏi và thực hiện đánh giá học viên; giám sát tiến độ học tập của các lớp học được phân công. Bên cạnh đó, nhóm này còn quản lý các diễn đàn thảo luận để giải đáp thắc mắc và xem các báo cáo phân tích chất lượng từ hệ thống AI.
- Quản trị hệ thống (Admin): Đảm nhận vai trò vận hành và bảo mật cho toàn bộ nền tảng. Các nhiệm vụ chính bao gồm quản lý người dùng và thực hiện phân quyền truy cập (RBAC); giám sát trạng thái hoạt động của các API và các Microservices để đảm bảo tính sẵn sàng cao. Admin cũng là người chịu trách nhiệm cấu hình các tham số cho mô hình AI Advisor để đảm bảo tính chính xác của các gợi ý học tập.

Hạ tầng kỹ thuật (Technical Infrastructure): Toàn bộ các nhóm chức năng trên được vận hành dựa trên sự kết hợp của các công nghệ hiện đại. Hệ thống sử dụng Apache Kafka để điều phối các sự kiện bất đồng bộ, Redis để tối ưu hóa tốc độ truy xuất thông qua bộ nhớ đệm và PostgreSQL để lưu trữ dữ liệu bền vững.

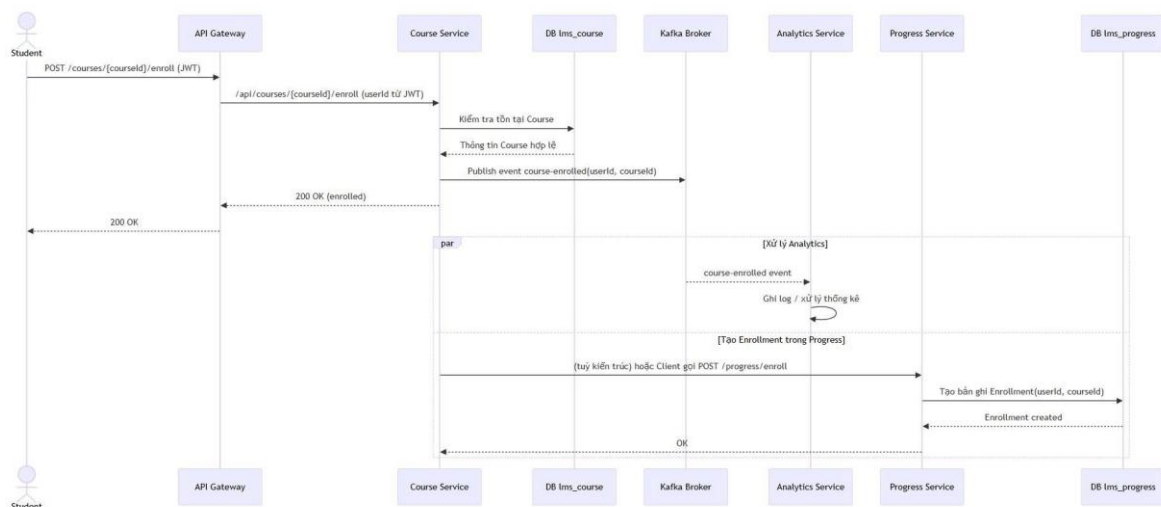
3.10. THIẾT KẾ SƠ ĐỒ TUẦN TỰ (SEQUENCE DIAGRAM)

3.10.1. Sequence Diagram: Đăng nhập



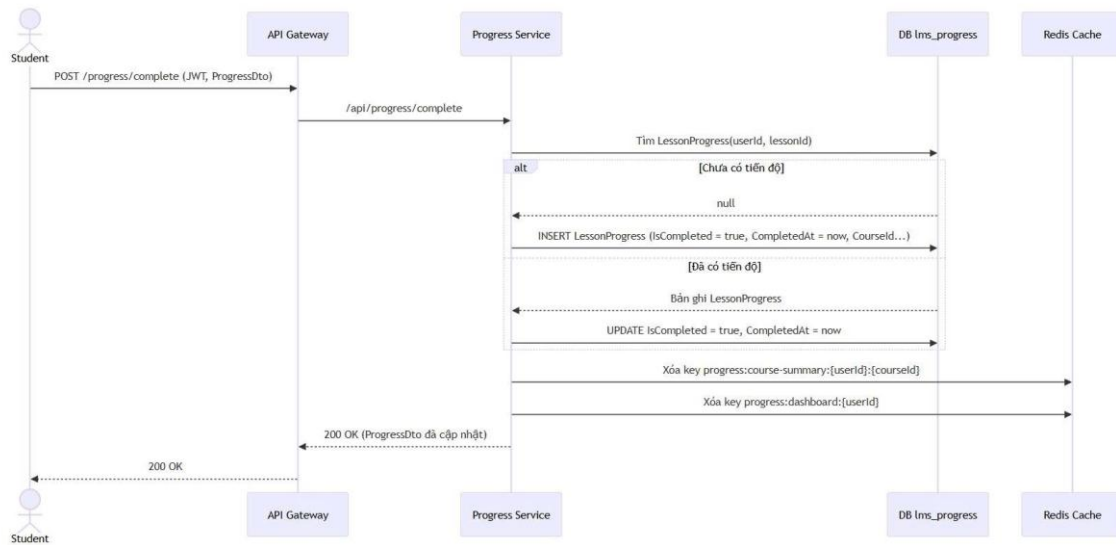
Hình 3.4. Sequence Diagram: Đăng nhập (Login qua Gateway → Auth)

3.10.2. Sequence Diagram: Học viên ghi danh khóa học



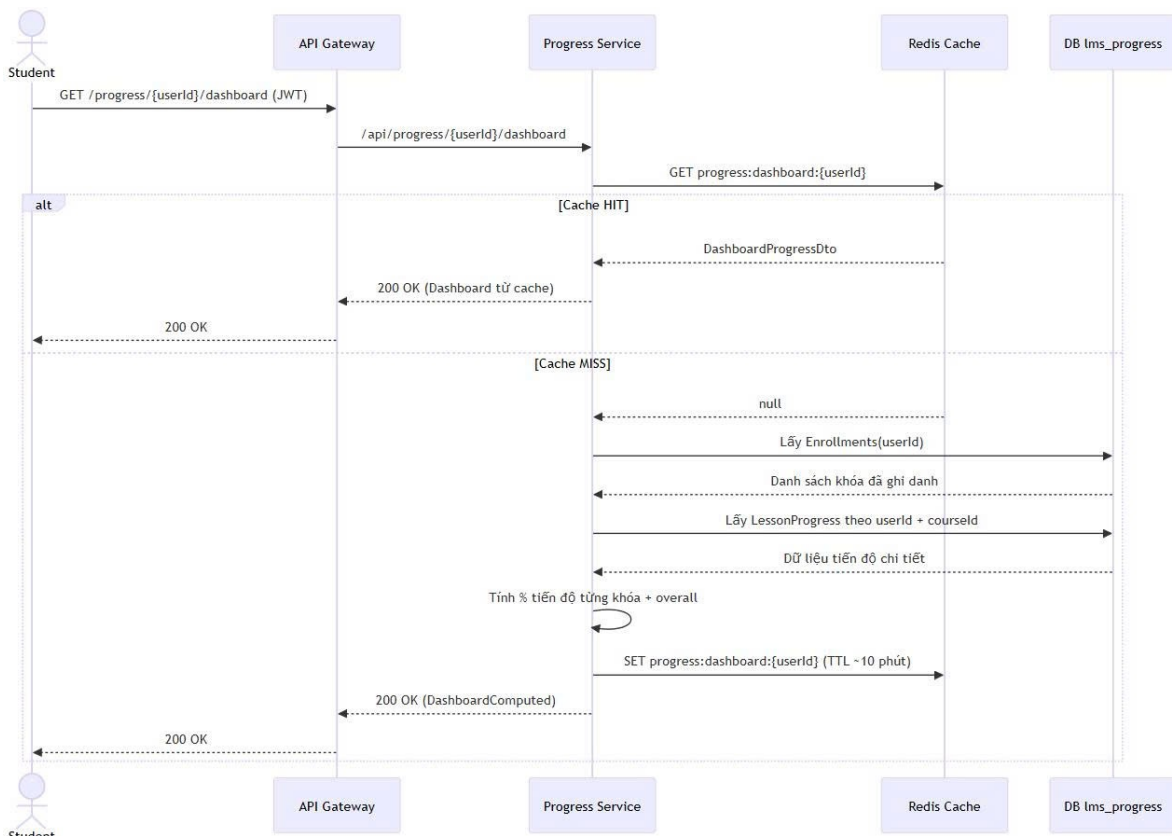
Hình 3.5. Sequence Diagram: Học viên ghi danh khóa học

3.10.3. Sequence Diagram: Hoàn thành bài học (CompleteLesson)



Hình 3.6. Sequence Diagram: Hoàn thành bài học (CompleteLesson)

3.10.4. Sequence Diagram: Xem Dashboard tiến độ



Hình 3.7. Sequence Diagram: Xem Dashboard tiến độ

3.11. THIẾT KẾ SƠ ĐỒ TRIỂN KHAI (DEPLOYMENT DIAGRAM)

Sơ đồ triển khai tập trung vào việc mô tả cách thức phân bổ các thành phần phần mềm lên hạ tầng ảo hóa. Hệ thống IntelligentLMS được thiết kế để triển khai dựa trên công nghệ Containerization của Docker, giúp đảm bảo tính đồng nhất giữa môi trường phát triển và môi trường thực tế.

Mỗi Microservice (.NET 8) sẽ được đóng gói trong một Docker Container riêng biệt, cho phép nhân bản nhanh chóng khi cần cân bằng tải. Các cụm cơ sở dữ liệu PostgreSQL và Redis được triển khai trên các nút lưu trữ độc lập để đảm bảo an toàn dữ liệu. Hệ thống Kafka Cluster cùng các dịch vụ Consumer được cấu hình chạy ngầm, kết nối với nhau qua mạng nội bộ bảo mật, tạo thành một hệ thống phân tán có khả năng chịu lỗi và sẵn sàng cao.

CHƯƠNG 4. TRIỂN KHAI VÀ ĐÁNH GIÁ

4.1. CÔNG NGHỆ SỬ DỤNG

4.1.1. Giới thiệu về ASP.NET Core

ASP.NET Core là một khung làm việc (framework) mã nguồn mở, đa nền tảng và có hiệu năng cực cao do Microsoft phát triển. Đây là công cụ chính được sử dụng để xây dựng các dịch vụ Backend (Microservices) trong dự án IntelligentLMS, cho phép hệ thống chạy mượt mà trên nhiều môi trường khác nhau từ Windows đến các container Docker trên Linux.

Khác với các ngôn ngữ kịch bản như PHP, ASP.NET Core là một ngôn ngữ biên dịch, giúp tối ưu hóa tốc độ thực thi và bảo mật mã nguồn tốt hơn. Trong dự án này, ASP.NET Core đóng vai trò xử lý các logic nghiệp vụ phức tạp thông qua các Controller, giao tiếp với cơ sở dữ liệu PostgreSQL thông qua Entity Framework Core và trả về dữ liệu dưới dạng JSON cho phía Client.

Mô hình hoạt động của Web API trong dự án:

Hệ thống sử dụng mô hình Web API (RESTful). Khi người dùng thao tác trên giao diện, một yêu cầu (Request) sẽ được gửi đến Gateway, sau đó Gateway điều phối đến ASP.NET Core Controller tương ứng để xử lý và phản hồi (Response) dữ liệu.

4.1.2. Giới thiệu về React và TypeScript

React là một thư viện JavaScript mã nguồn mở được Facebook phát triển, chuyên dùng để xây dựng giao diện người dùng (UI). React cho phép xây dựng các ứng dụng đơn trang (SPA) mạnh mẽ, nơi các thành phần giao diện (Components) được quản lý độc lập và tái sử dụng linh hoạt. Trong dự án này, React đảm nhận vai trò hiển thị Dashboard học tập, biểu đồ tiến độ và các tương tác thời gian thực của học viên.

Để tăng tính chặt chẽ, dự án kết hợp React cùng với TypeScript. TypeScript là một bản nâng cấp của JavaScript, bổ sung các kiểu dữ liệu tĩnh (Static Typing), giúp phát hiện lỗi ngay trong quá trình viết code và làm cho mã nguồn trở nên dễ đọc, dễ bảo trì hơn trong các hệ thống lớn.

Ứng dụng trong IntelligentLMS:

React sử dụng cơ chế Virtual DOM để chỉ cập nhật những phần dữ liệu thay đổi trên màn hình mà không cần tải lại toàn bộ trang web. Ví dụ, khi dữ liệu tiến độ từ Backend trả về con số 30%, React sẽ chỉ thay đổi thanh trạng thái và con số hiển thị, giúp trải nghiệm người dùng trở nên cực kỳ mượt mà..

4.1.3. Hệ quản trị cơ sở dữ liệu PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ (RDBMS) mã nguồn mở mạnh mẽ và tiên tiến nhất hiện nay. Nó nổi tiếng với độ tin cậy, tính toàn vẹn của dữ liệu và khả năng mở rộng tốt. Trong kiến trúc Microservices của IntelligentLMS, PostgreSQL đóng vai trò là "kho lưu trữ vĩnh viễn" cho các dịch vụ như quản lý khóa học và tiến độ học tập.

Hệ thống sử dụng PostgreSQL để lưu trữ các thông tin có cấu trúc phức tạp như UUID của người dùng, lộ trình bài học và lịch sử học tập. Các ràng buộc dữ liệu nghiêm ngặt trong PostgreSQL giúp đảm bảo rằng con số tiến độ (ví dụ 30%) luôn được tính toán chính xác dựa trên số bài học đã hoàn thành thực tế.

Mô hình lưu trữ:

Dữ liệu được tổ chức thành các bảng (Tables) có quan hệ chặt chẽ với nhau. Thông qua Entity Framework Core từ phía Backend, các câu lệnh truy vấn sẽ được thực thi để trích xuất thông tin từ các bảng như Enrollments hay CourseProgress nhằm cung cấp dữ liệu chính xác nhất cho người dùng...

4.1.4. Hệ thống lưu trữ dữ liệu Redis

Redis (Remote Dictionary Server) là một hệ thống lưu trữ cấu trúc dữ liệu trong bộ nhớ (In-memory), thường được sử dụng làm cơ sở dữ liệu NoSQL, bộ nhớ đệm (Cache) và môi trường truyền tin. Vì hoạt động hoàn toàn trên RAM, Redis có tốc độ truy xuất cực nhanh, lên đến hàng triệu yêu cầu mỗi giây.

Trong dự án IntelligentLMS, Redis đóng vai trò là lớp bộ nhớ đệm trung gian giữa Backend và PostgreSQL. Thay vì bắt PostgreSQL phải thực hiện các truy vấn nặng nề liên tục, các dữ liệu thường xuyên được truy cập như phiên đăng nhập (Session) hoặc trạng thái tiến độ tạm thời sẽ được lưu vào Redis để phản hồi ngay lập tức cho phía Client.

Mô hình Caching:

Khi React gửi yêu cầu lấy tiền độ, Backend sẽ kiểm tra trong Redis trước. Nếu dữ liệu đã có (Cache Hit), nó sẽ trả về ngay lập tức. Nếu chưa có (Cache Miss), Backend mới truy vấn vào PostgreSQL, sau đó lưu kết quả vào Redis cho các lần sử dụng sau.

4.1.5. Công nghệ ảo hóa Docker và Docker Compose

Docker là một nền tảng mã nguồn mở cho phép đóng gói ứng dụng và tất cả các thành phần phụ thuộc vào trong một đơn vị phần mềm gọi là Container. Container giúp đảm bảo rằng ứng dụng sẽ chạy giống hệt nhau trên bất kỳ môi trường nào, từ máy tính cá nhân của Diey đến máy chủ triển khai thực tế.

Docker Compose là công cụ đi kèm giúp định nghĩa và quản lý các ứng dụng Docker đa container. Trong IntelligentLMS, Docker Compose cho phép khởi chạy đồng thời toàn bộ hệ thống bao gồm gateway-1, auth-service, course-service, lms_postgres và redis chỉ bằng một tệp cấu hình duy nhất.

Lợi ích đối với dự án:

Việc sử dụng Docker giúp cô lập các dịch vụ, tránh xung đột thư viện giữa các Microservices và giúp việc quản lý hạ tầng trở nên đơn giản, nhất quán.

4.1.6. Công nghệ ứng dụng

Hệ thống IntelligentLMS tích hợp một hệ sinh thái công nghệ hiện đại nhằm đảm bảo hiệu năng và khả năng mở rộng linh hoạt. Tầng xử lý phía máy chủ được phát triển trên nền tảng .NET 8 với ASP.NET Core để xây dựng các dịch vụ Microservices hoạt động độc lập. Giao diện người dùng sử dụng thư viện React kết hợp cùng ngôn ngữ TypeScript để tối ưu hóa trải nghiệm tương tác và tính chặt chẽ của mã nguồn trên Dashboard học tập.

Dữ liệu hệ thống được quản lý bởi hệ quản trị PostgreSQL cho các thông tin quan trọng và Redis đóng vai trò bộ nhớ đệm giúp tăng tốc độ phản hồi cho các yêu cầu truy xuất thường xuyên. Toàn bộ quá trình điều phối yêu cầu và giao tiếp giữa các dịch vụ được thực hiện thông qua API Gateway và hệ thống truyền tin bất đồng bộ Apache Kafka.

Cuối cùng, công nghệ Docker và Docker Compose được ứng dụng để đóng gói và triển khai toàn bộ hạ tầng một cách nhất quán, đảm bảo sự đồng bộ từ môi trường phát triển đến vận hành thực tế. Ngoài ra, hệ thống còn tích hợp module AI Advisor để thực hiện các phân tích dữ liệu hành vi và cung cấp lộ trình học tập cá nhân hóa cho học viên.

4.1.7. Đánh giá lựa chọn công nghệ

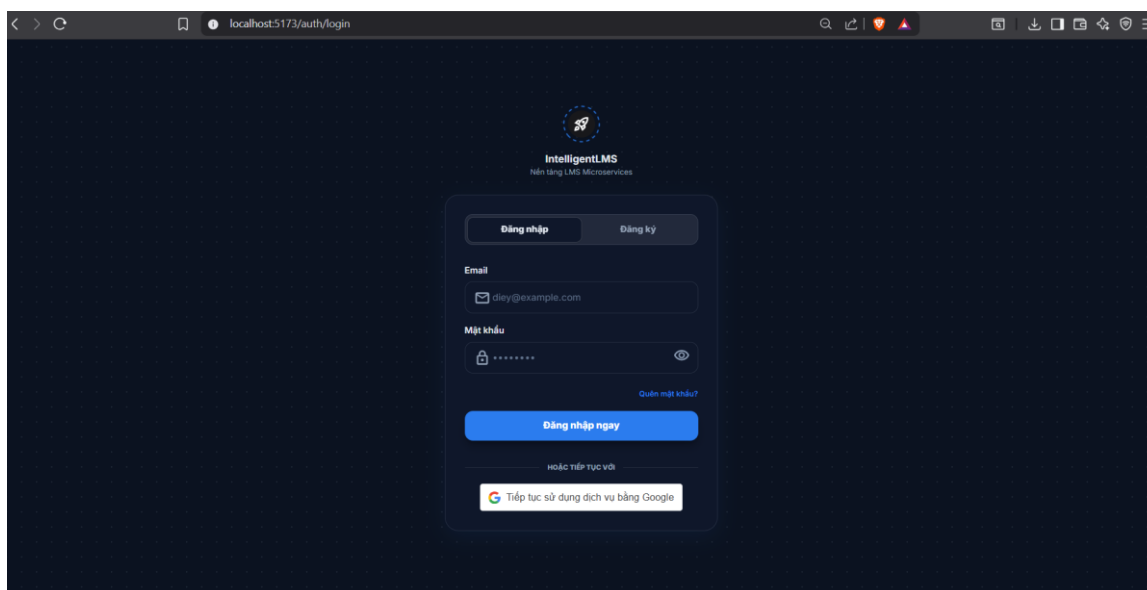
Bảng 4.1. Tổng hợp ưu và nhược điểm của các công nghệ áp dụng

STT	Tên nghiên cứu / Mô hình	Công nghệ áp dụng	Ưu điểm	Hạn chế
1	Kiến trúc Microservices	.NET 8, Docker	Mở rộng linh hoạt, triển khai độc lập từng service	Quản lý phức tạp, cần DevOps tốt
2	API Gateway Pattern	Gateway (.NET), JWT	Tập trung bảo mật, quản lý routing dễ dàng	Tăng độ phức tạp hệ thống
3	RESTful Web API	ASP.NET Core	Hiệu năng cao, dễ tích hợp frontend	Phụ thuộc thiết kế API chuẩn
4	SPA với React	React + TypeScript	Giao diện mượt, cập nhật realtime (Virtual DOM)	SEO hạn chế nếu không cấu hình SSR
5	Cơ sở dữ liệu quan hệ	PostgreSQL	Toàn vẹn dữ liệu cao, hỗ trợ truy vấn mạnh	Tốn tài nguyên khi truy vấn lớn
6	Caching In-memory	Redis	Tốc độ truy xuất cực nhanh, giảm tải DB	Dữ liệu lưu trên RAM, cần cấu hình backup
7	Event-Driven Architecture	Apache Kafka	Xử lý bất đồng bộ, mở rộng tốt	Triển khai và vận hành phức tạp

8	Containerization	Docker + Docker Compose	Môi trường nhất quán, dễ triển khai	Cần tài nguyên hệ thống đủ mạnh
---	------------------	-------------------------------	--	---------------------------------------

4.2. DEMO HỆ THỐNG

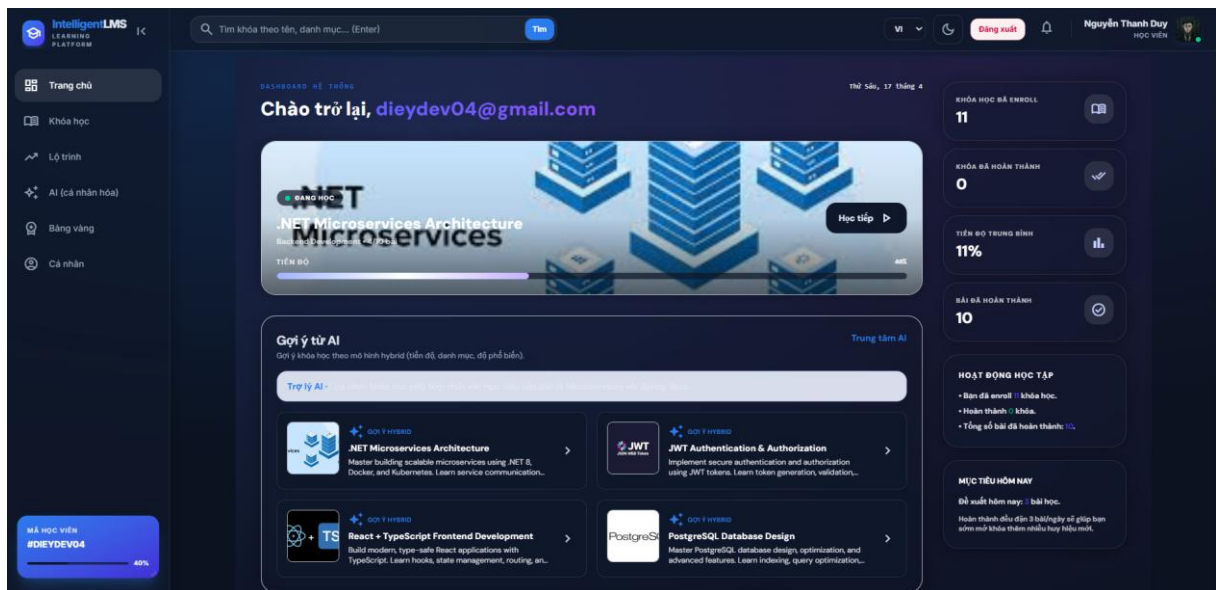
4.2.1. Giao diện đăng nhập



Hình 4.1. Giao diện đăng nhập

Giao diện đăng nhập của hệ thống IntelligentLMS được thiết kế theo phong cách Dark Mode hiện đại, sử dụng tông màu chủ đạo xanh - đen cùng họa tiết lưới (grid) tạo cảm giác công nghệ và chuyên nghiệp. Bố cục được tối giản và căn giữa, giúp người dùng tập trung vào khung đăng nhập chính bao gồm các trường thông tin rõ ràng (Email, Mật khẩu) và nút tác vụ nổi bật. Đặc biệt, hệ thống tích hợp linh hoạt giữa phương thức đăng nhập truyền thống và xác thực qua Google, kết hợp với các tính năng bổ sung như ẩn/hiện mật khẩu và chuyển đổi nhanh giữa Đăng nhập/Đăng ký, nhằm tối ưu hóa trải nghiệm người dùng (UX) và đảm bảo quy trình xác thực diễn ra nhanh chóng, mượt mà.

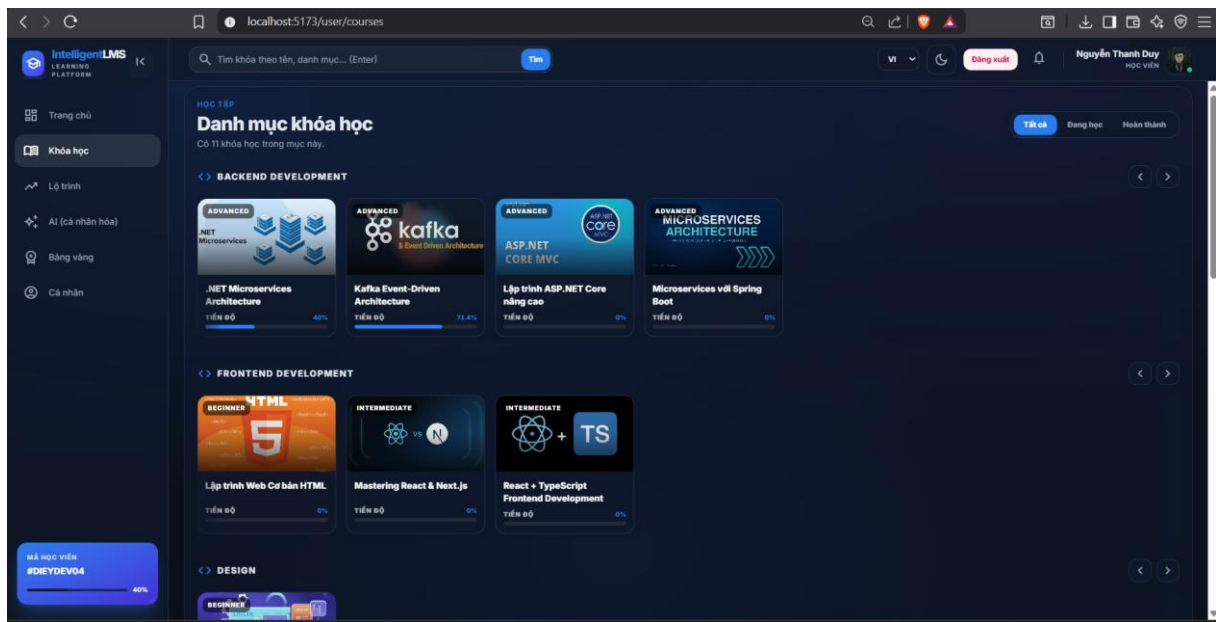
4.2.2. Giao diện trang chủ



Hình 4.2. Giao diện trang chủ

Giao diện Dashboard của IntelligentLMS được thiết kế dưới dạng một trung tâm điều khiển cá nhân hóa, giúp người dùng dễ dàng quản lý lộ trình học tập và theo dõi tiến độ thời gian thực. Bố cục trang bao gồm hệ thống thanh điều hướng bên trái (Sidebar) trực quan và khu vực nội dung chính được phân bổ khoa học: phía trên cùng hiển thị lời chào định danh kèm trạng thái khóa học đang theo dõi gần nhất; khu vực trung tâm tập trung vào tính năng "Gợi ý từ AI" giúp đề xuất các khóa học chuyên sâu dựa trên dữ liệu cá nhân; phía bên phải là cột thống kê chi tiết các chỉ số như số lượng khóa học đã đăng ký, tiến độ trung bình và hoạt động học tập gần đây. Toàn bộ giao diện duy trì ngôn ngữ thiết kế Dark Mode đồng nhất với các thẻ (cards) được bo góc hiện đại, tạo không gian làm việc tập trung và làm nổi bật các thành phần dữ liệu quan trọng, hỗ trợ tối đa người dùng trong việc định hướng học tập hàng ngày.

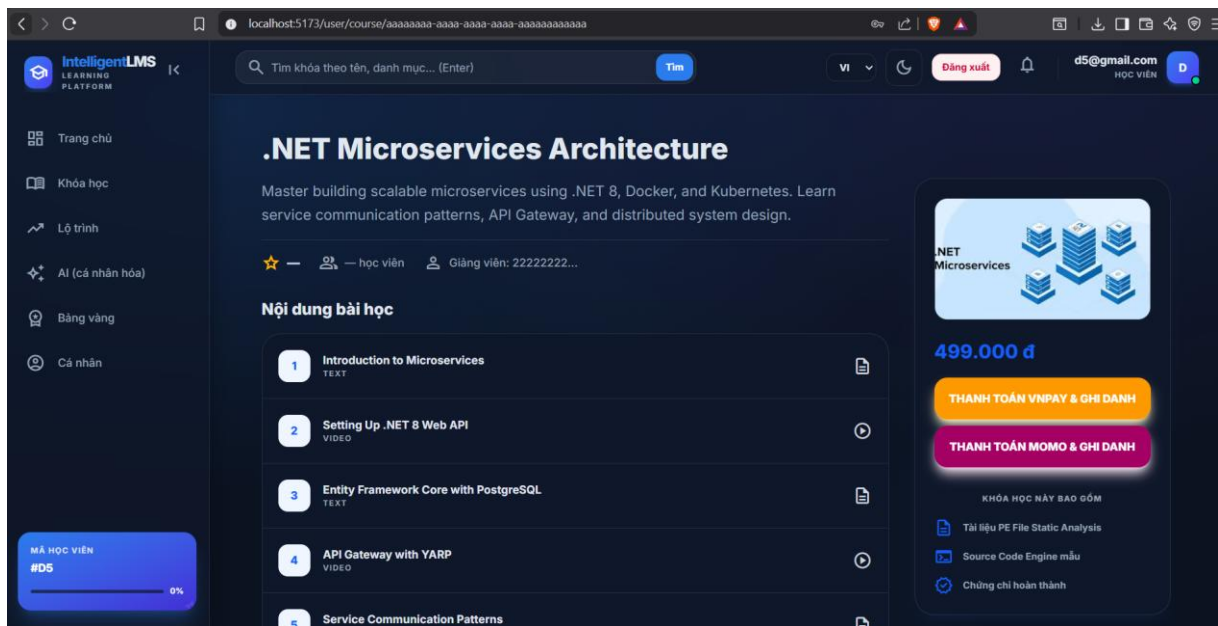
4.2.3. Giao diện khóa học



Hình 4.3. Giao diện khóa học

Giao diện Danh mục khóa học của IntelligentLMS được tổ chức theo cấu trúc phân cấp khoa học, cho phép người dùng dễ dàng tìm kiếm và quản lý các nội dung học tập. Các khóa học được phân loại rõ rệt theo từng lĩnh vực chuyên môn như *Backend Development*, *Frontend Development* và *Design* dưới dạng các hàng cuộn ngang (horizontal scroll) tiết kiệm không gian. Mỗi thẻ khóa học (course card) cung cấp đầy đủ thông tin quan trọng bao gồm: hình ảnh định danh công nghệ, trình độ (Advanced, Intermediate, Beginner), tên khóa học và thanh tiến độ (progress bar) trực quan giúp học viên nắm bắt nhanh chóng quá trình học tập của bản thân. Phía trên cùng tích hợp bộ lọc trạng thái (Tất cả, Đang học, Hoàn thành) cùng thanh tìm kiếm thông minh, tạo nên một hệ thống quản lý học tập mạch lạc, hỗ trợ đắc lực cho việc cá nhân hóa lộ trình phát triển kỹ năng của người dùng.

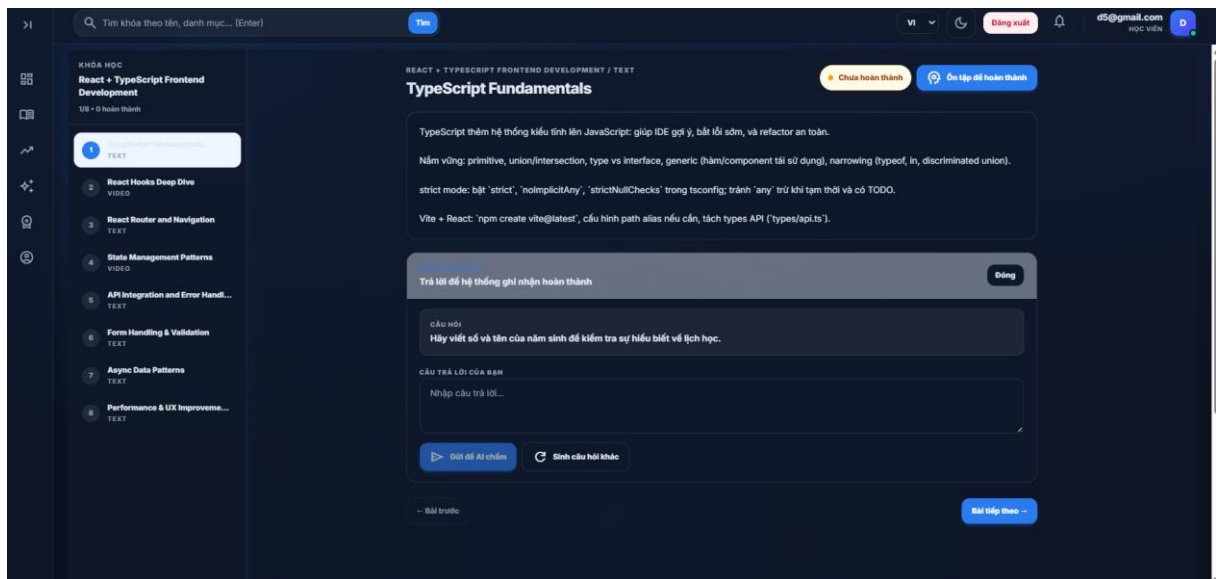
4.2.4. Giao diện thanh toán & ghi danh



Hình 4.4. Giao diện thanh toán & ghi danh

Giao diện chi tiết khóa học được thiết kế tập trung vào việc cung cấp thông tin đầy đủ và thúc đẩy hành động đăng ký của học viên. Phần bên trái trình bày tiêu đề khóa học nổi bật cùng mô tả ngắn gọn về giá trị cốt lõi (như .NET 8, Docker, Kubernetes) và danh sách lộ trình bài học được đánh số thứ tự, phân loại rõ định dạng nội dung (Text/Video). Phía bên phải là cột thông tin thanh toán được cố định trực quan, hiển thị giá tiền và tích hợp các phương thức thanh toán điện tử phổ biến (VNPAY, MoMo) qua các nút bấm có hiệu ứng đồ bóng phát sáng (glow effect) thu hút sự chú ý. Ngoài ra, các lợi ích đi kèm như tài liệu độc quyền, mã nguồn mẫu và chứng chỉ hoàn thành cũng được liệt kê rõ ràng bằng hệ thống icon chuyên nghiệp, giúp gia tăng mức độ tin cậy và giá trị chuyển đổi cho nền tảng.

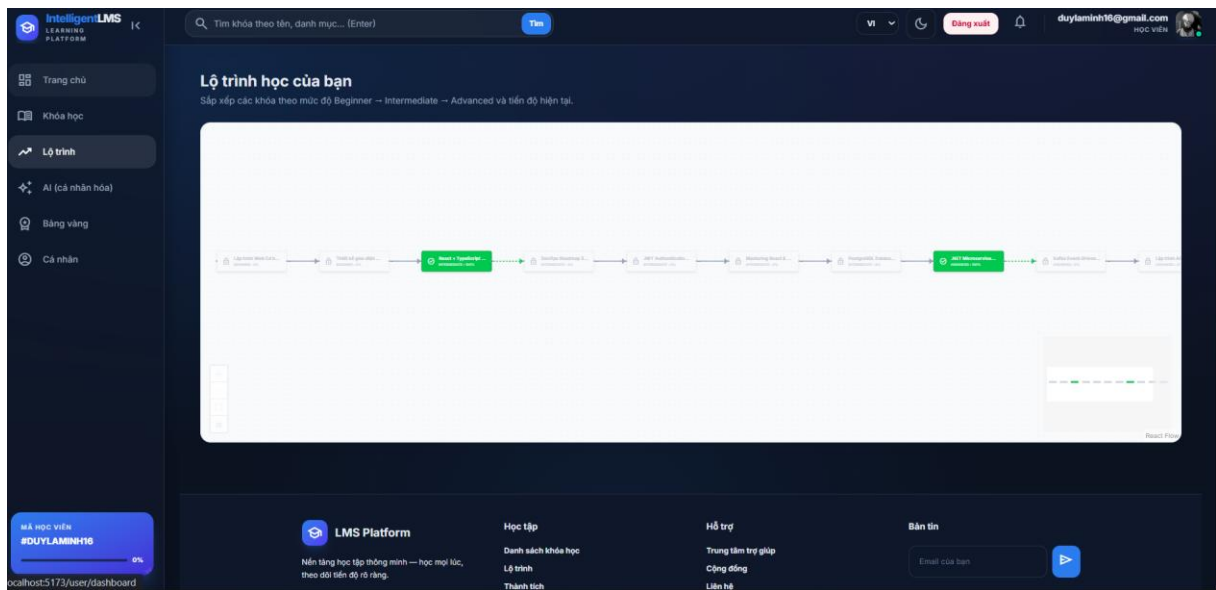
4.2.5. Giao diện nội dung bài học và kiểm tra kiến thức với AI



Hình 4.5. Giao diện học và kiểm tra kiến thức với AI

Giao diện học tập tập trung vào trải nghiệm đọc và tương tác trực tiếp, giúp học viên dễ dàng tiếp thu kiến thức chuyên sâu. Cấu trúc trang bao gồm thanh danh mục bài học bên trái (Lesson Sidebar) giúp theo dõi tiến độ tổng thể, và khu vực nội dung chính hiển thị các kiến thức trọng tâm được trình bày mạch lạc. Điểm nhấn đặc biệt của giao diện là tính năng "Tư vấn cùng AI", yêu cầu học viên trả lời câu hỏi kiểm tra để ghi nhận hoàn thành bài học. Hệ thống cho phép học viên nhập câu trả lời trực tiếp và gửi để AI chấm điểm thời gian thực hoặc yêu cầu sinh câu hỏi khác, tạo nên một vòng lặp phản hồi tích cực. Sự kết hợp giữa nội dung lý thuyết và bộ công cụ đánh giá thông minh ngay trong một màn hình không chỉ nâng cao tính tương tác mà còn đảm bảo người dùng thực sự nắm vững kiến thức trước khi chuyển sang bài học kế tiếp.

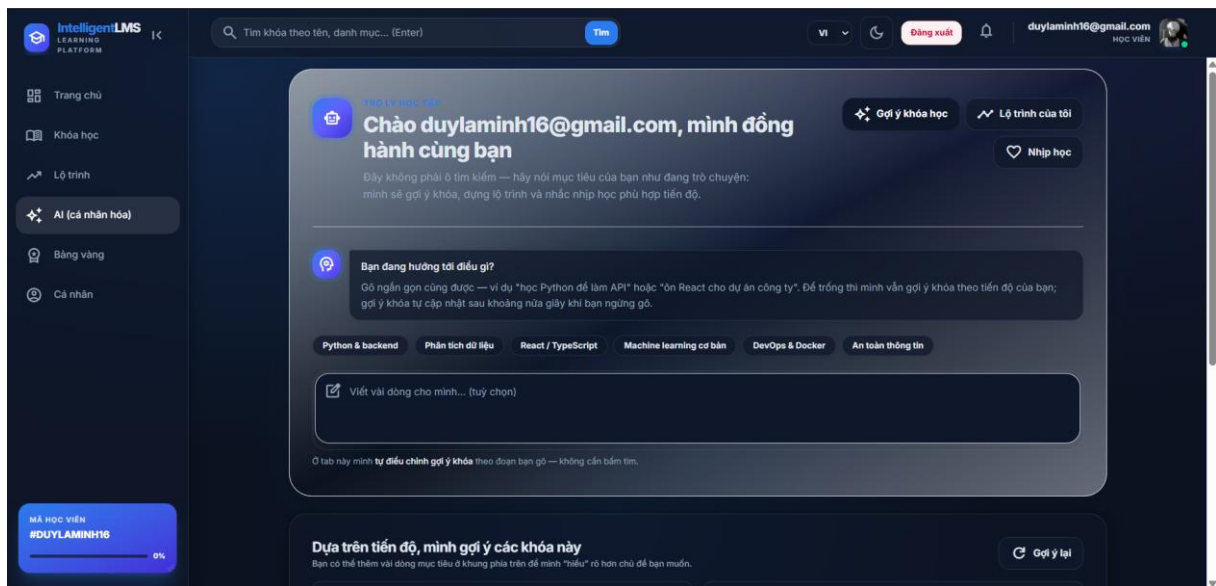
4.2.6. Giao diện lộ trình học tập



Hình 4.6. Giao diện lộ trình học tập

Giao diện Lộ trình học tập của IntelligentLMS cung cấp cái nhìn tổng quan và định hướng phát triển kỹ năng dài hạn cho học viên thông qua sơ đồ dòng chảy (flowchart) trực quan. Các khóa học được sắp xếp theo trình tự logic từ cơ bản (Beginner) đến nâng cao (Advanced), giúp người dùng hiểu rõ mối liên hệ giữa các khối kiến thức. Hệ thống sử dụng các trạng thái màu sắc và biểu tượng để phân biệt: các bài học đã hoàn thành được làm nổi bật với sắc xanh dương/xanh lá, trong khi các bài học tiếp theo được hiển thị dưới dạng khóa (locked) hoặc mờ dần, tạo động lực chinh phục theo từng giai đoạn. Với tính năng điều hướng linh hoạt (zoom in/out) trên một không gian làm việc vô hạn, giao diện này không chỉ là một công cụ quản lý mà còn là bản đồ dẫn đường cá nhân hóa, giúp học viên duy trì sự tập trung và kiên trì trong suốt hành trình học tập.

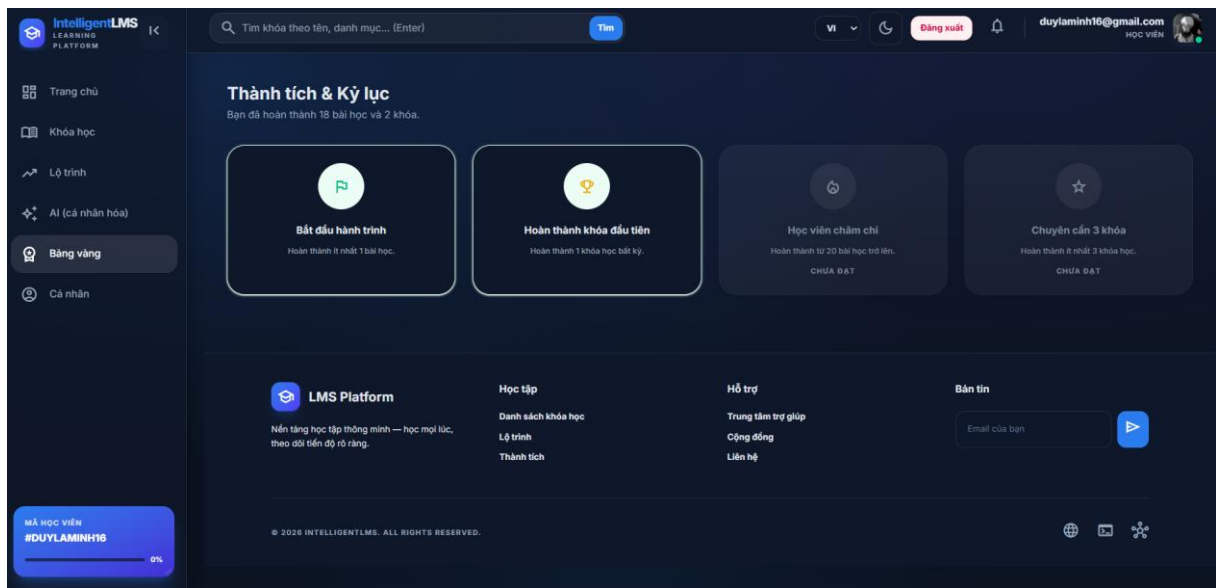
4.2.7. Giao diện AI hỗ trợ



Hình 4.7. Giao diện AI hỗ trợ

Giao diện AI (cá nhân hóa) đóng vai trò là hạt nhân thông minh của hệ thống, cung cấp một không gian tương tác hội thoại trực tiếp giữa người dùng và trợ lý ảo. Điểm nổi bật của màn hình này là khung nhập liệu thông minh cho phép học viên chia sẻ mục tiêu học tập cụ thể, từ đó hệ thống sẽ tự động phân tích và đề xuất lộ trình, khóa học phù hợp dựa trên từ khóa và tiến độ thực tế. Giao diện được tối ưu với các nhãn gợi ý nhanh (tags) như *Python & Backend*, *React/TypeScript*, *DevOps*... giúp người dùng định hình yêu cầu chỉ bằng một cú click. Với cơ chế tự động cập nhật gợi ý theo thời gian thực mà không cần tải lại trang, tính năng này không chỉ tạo cảm giác hiện đại mà còn mang lại trải nghiệm học tập mang tính định hướng cao, hỗ trợ tối đa việc cá nhân hóa kiến thức cho từng học viên.

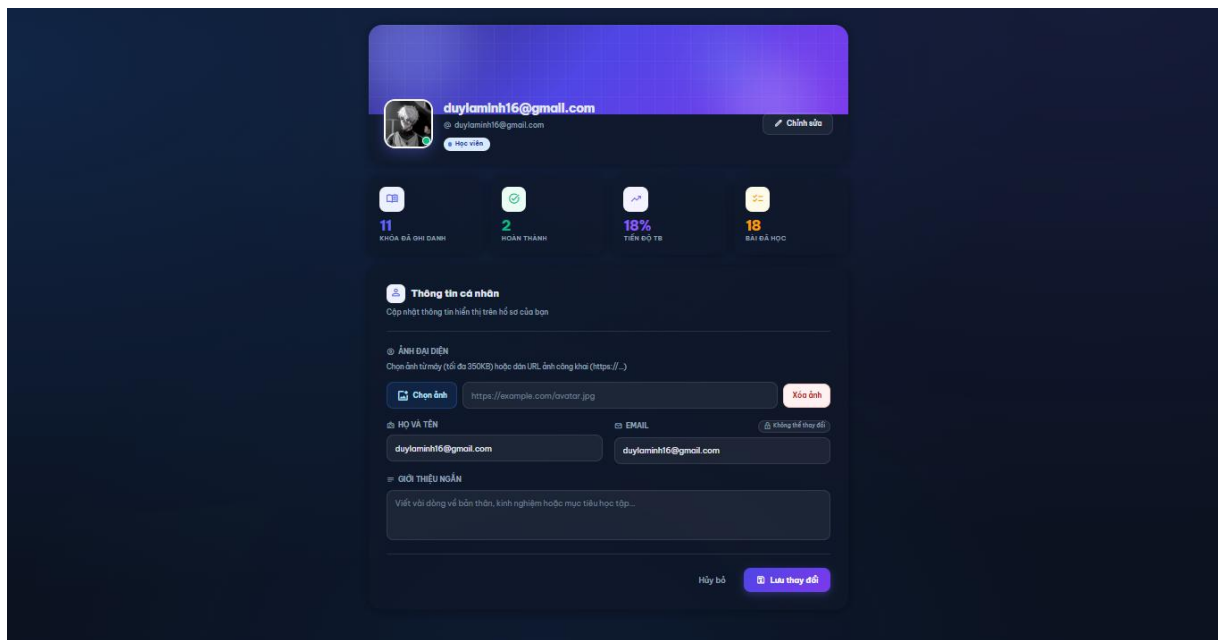
4.2.8. Giao diện thành tích & kỷ lục



Hình 4.8. Giao diện bảng vàng

Giao diện Bảng vàng được thiết kế nhằm mục đích trò chơi hóa (gamification) quá trình học tập, giúp tăng cường động lực và sự gắn kết của người dùng thông qua việc ghi nhận các cột mốc quan trọng. Trung tâm của màn hình là hệ thống các thẻ thành tích (achievement cards) với thiết kế huy hiệu trực quan; các thành tích đã đạt được sẽ hiển thị trạng thái sáng nổi bật kèm mô tả ngắn, trong khi các mục tiêu chưa đạt được sẽ hiển thị ở dạng làm mờ (grayscale) để kích thích tinh thần chinh phục. Phía cuối trang tích hợp một Footer (chân trang) đầy đủ thông tin về hệ sinh thái nền tảng, các liên kết hỗ trợ và đăng ký bản tin, tạo nên một kết cấu trang hoàn chỉnh, chuyên nghiệp. Tính năng này không chỉ giúp học viên tự hào về lộ trình đã đi qua mà còn đóng vai trò như một bảng hệ thống kỷ lục cá nhân, thúc đẩy tinh thần học tập bền bỉ trong môi trường số.

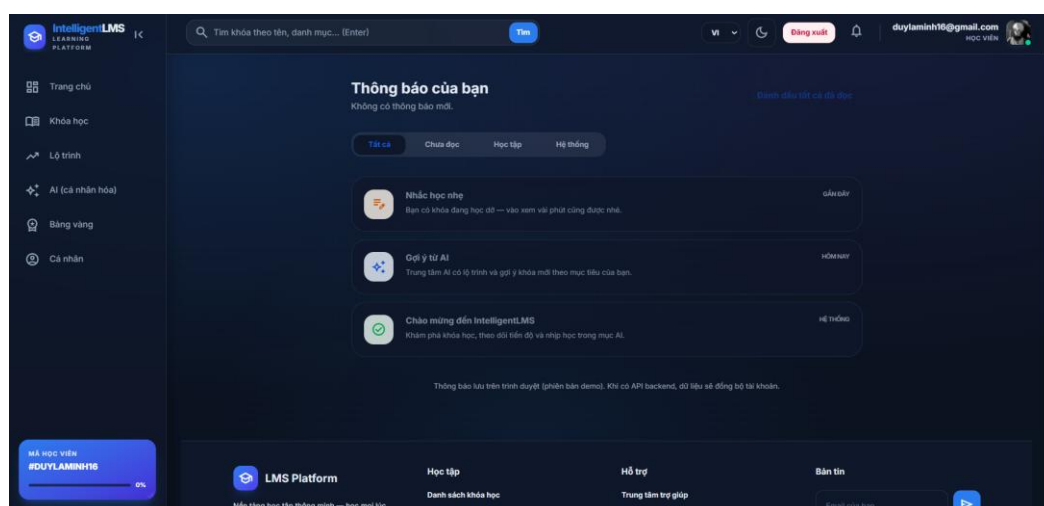
4.2.9. Giao diện Profile



Hình 4.9. Giao diện profile

Giao diện Hồ sơ cá nhân của IntelligentLMS được thiết kế tập trung vào tính cá nhân hóa và quản trị thông tin người dùng một cách trực quan. Phía trên cùng là khu vực định danh nổi bật với ảnh đại diện (avatar) và ảnh bìa (cover photo) mang phong cách gradient hiện đại, đi kèm với các thẻ thống kê nhanh về tổng số khóa học, tiến độ trung bình và số bài đã học. Phần nội dung chính là biểu mẫu "Thông tin cá nhân" cho phép người dùng chủ động cập nhật hình ảnh, họ tên và phần giới thiệu bản thân một cách dễ dàng thông qua các trường nhập liệu tối giản. Giao diện duy trì sự nhất quán về thẩm mỹ với các nút chức năng "Lưu thay đổi" và "Hủy bỏ" được bố trí hợp lý, tạo ra một không gian quản lý tài khoản vừa đảm bảo tính thẩm mỹ, vừa tối ưu hóa tính tiện dụng cho học viên.

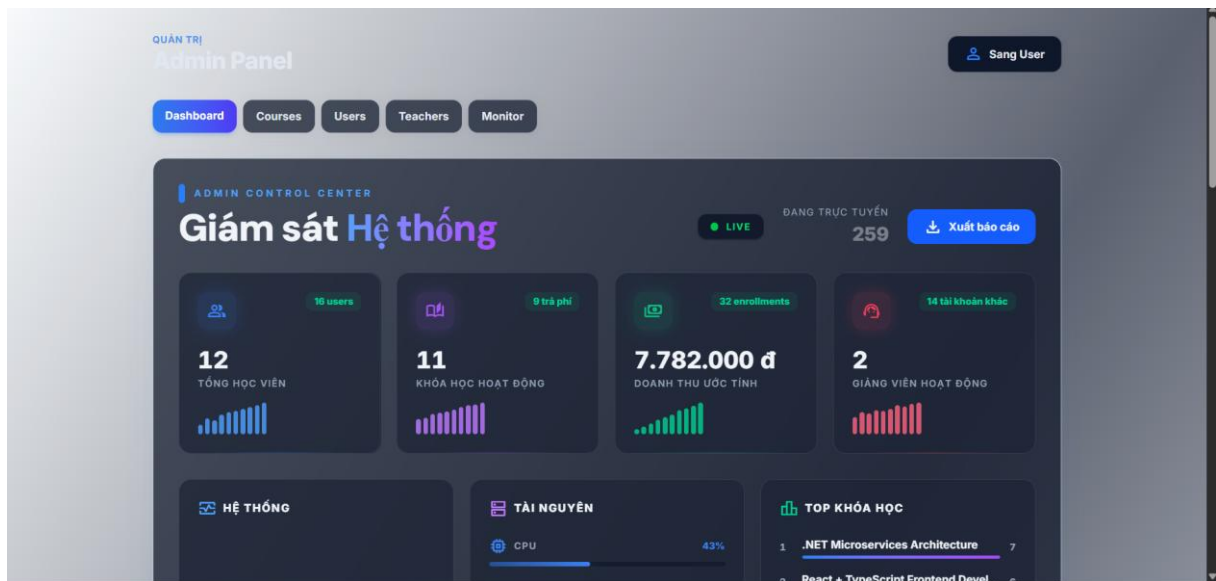
4.2.10. Giao diện thông báo



Hình 4.10. Giao diện thông báo

Giao diện Thông báo được thiết kế tối giản và trực quan, đóng vai trò là kênh kết nối thông tin liên tục giữa hệ thống và người dùng. Danh sách thông báo được phân loại thông minh theo các tab chức năng như *Tất cả*, *Chưa đọc*, *Học tập* và *Hệ thống*, giúp học viên dễ dàng quản lý và ưu tiên các luồng thông tin quan trọng. Mỗi dòng thông báo đều đi kèm với các icon màu sắc đặc trưng đại diện cho từng loại nội dung (như nhắc nhở học tập, gợi ý từ AI hay thông báo hệ thống), cùng với mốc thời gian nhận rõ ràng. Phía trên cùng tích hợp tính năng "Đánh dấu tất cả đã đọc" giúp tối ưu hóa thao tác quản lý dữ liệu. Toàn bộ bố cục không chỉ đảm bảo tính nhất quán về mặt thẩm mỹ mà còn hỗ trợ người dùng luôn cập nhật được lộ trình học tập và các thay đổi quan trọng từ nền tảng một cách nhanh chóng nhất.

4.2.11. Giao diện trang chủ Admin



Hình 4.11. Giao diện trang chủ Admin

Giao diện Quản trị của IntelligentLMS được thiết kế tập trung vào khả năng giám sát toàn diện và quản lý dữ liệu tập trung. Bố cục trang bao gồm một hệ thống menu điều hướng nhanh (Dashboard, Courses, Users, Teachers, Monitor) và khu vực hiển thị các chỉ số vận hành quan trọng dưới dạng thẻ (cards) thống kê trực quan. Các chỉ số như tổng số học viên, số khóa học đang hoạt động, doanh thu ước tính và số lượng giảng viên được trình bày kèm theo biểu đồ tăng trưởng thu nhỏ, giúp quản trị viên nắm bắt nhanh xu hướng của hệ thống. Bên cạnh đó, tính năng giám sát trạng thái trực tuyến (Live) theo thời gian thực và theo dõi tài nguyên hệ thống (CPU, RAM) được tích hợp ngay tại trung tâm điều khiển, kết hợp cùng các công cụ xuất báo cáo và quản trị nhanh, tạo nên một môi trường quản lý mạnh mẽ, chặt chẽ và chuyên nghiệp.

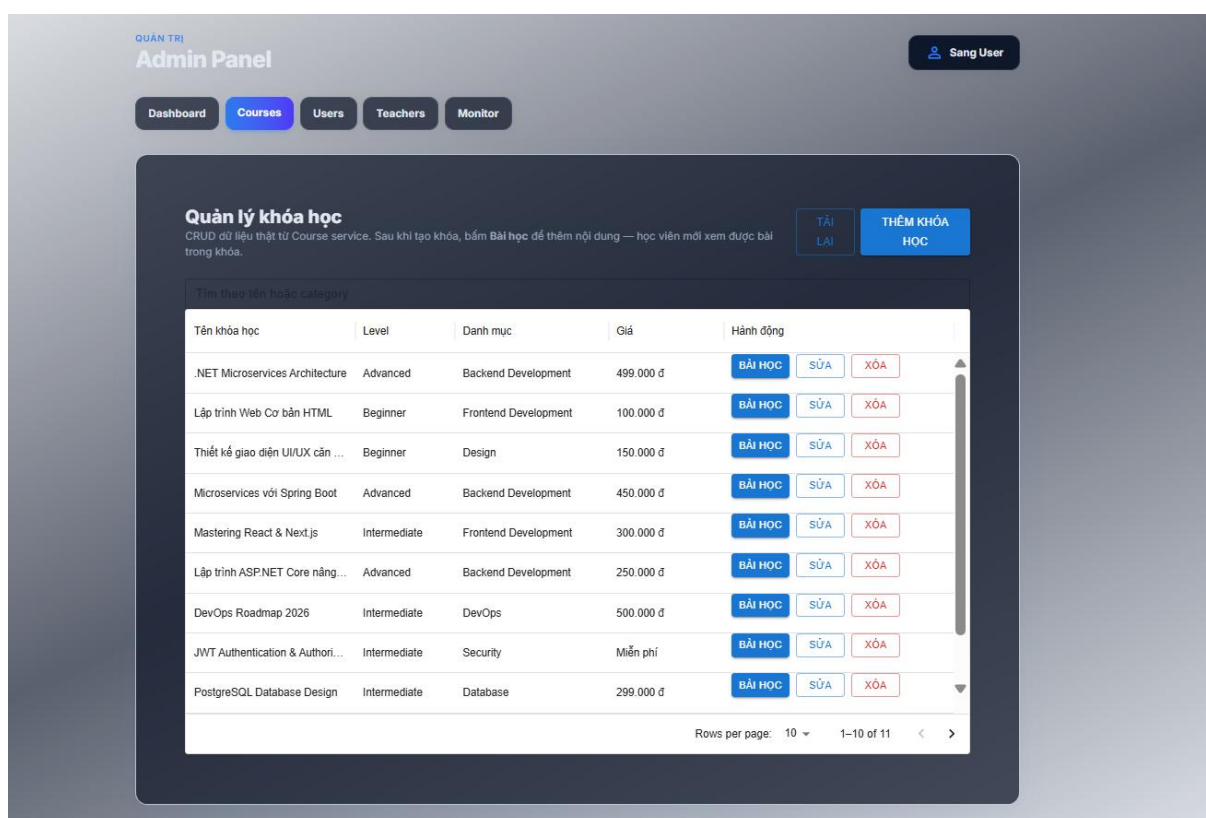
4.2.12. Giao diện trạng thái máy chủ Admin



Hình 4.12. Giao diện giám sát hạ tầng

Giao diện giám sát hạ tầng của IntelligentLMS được thiết kế như một trung tâm điều khiển kỹ thuật chuyên sâu, cho phép theo dõi sức khỏe hệ thống theo thời gian thực với độ chính xác cao. Khu vực phía trên hiển thị chi tiết các thông số phần cứng cốt lõi bao gồm mức độ sử dụng CPU, dung lượng RAM và trạng thái lưu trữ SSD thông qua các biểu đồ tròn (gauge chart) và đồ thị đường (line chart) sinh động. Phía dưới là các khối thông tin về hiệu suất mạng (Network I/O) với các chỉ số về độ trễ (latency) và lưu lượng truy cập qua Gateway. Đặc biệt, giao diện tích hợp bảng theo dõi trạng thái các Microservices (như Gateway, Auth, Course API, Progress) kèm theo cổng (port) hoạt động và đèn báo tín hiệu (status indicator), giúp quản trị viên hệ thống nhanh chóng phát hiện và xử lý các sự cố kỹ thuật, đảm bảo nền tảng luôn vận hành ổn định và thông suốt.

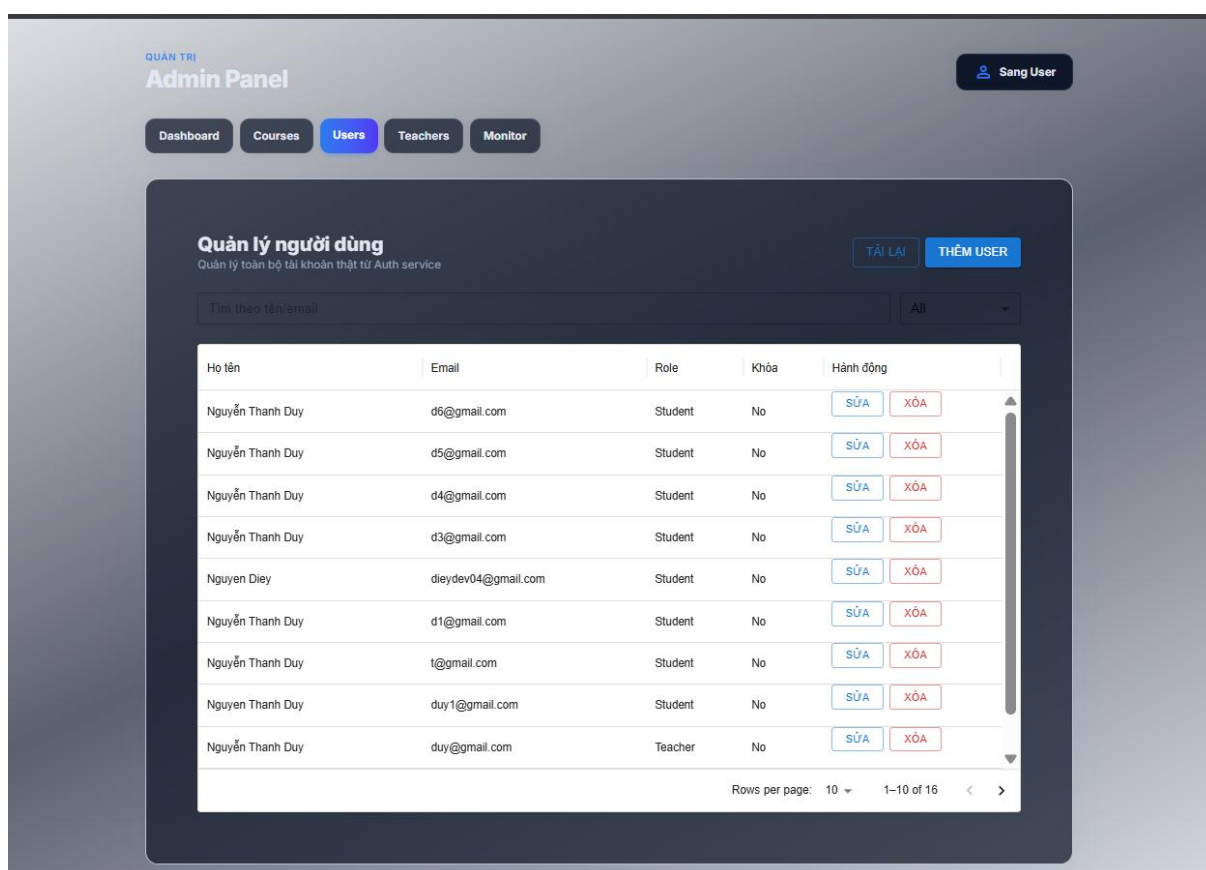
4.2.13. Giao diện quản lý khóa học



Hình 4.13. Giao diện quản lý khóa học

Giao diện Quản lý khóa học được thiết kế theo dạng bảng (table) chuyên nghiệp, tối ưu hóa cho các thao tác quản trị dữ liệu lớn một cách hệ thống. Danh sách khóa học được hiển thị chi tiết với các cột thông tin quan trọng bao gồm: hình ảnh đại diện, tên khóa học, danh mục phân loại, cấp độ bài học và trạng thái hiển thị. Hệ thống tích hợp các bộ lọc thông minh theo danh mục và trạng thái, cùng thanh tìm kiếm thời gian thực giúp người quản lý nhanh chóng định vị nội dung cần chỉnh sửa. Tại mỗi dòng dữ liệu, các nút tác vụ (Xem, Sửa, Xóa) được bố trí gọn gàng, kết hợp cùng tính năng "Thêm khóa học mới" nổi bật phía trên, tạo nên một quy trình quản lý nội dung khép kín, minh bạch và hiệu quả cao trong việc vận hành nền tảng giáo dục.

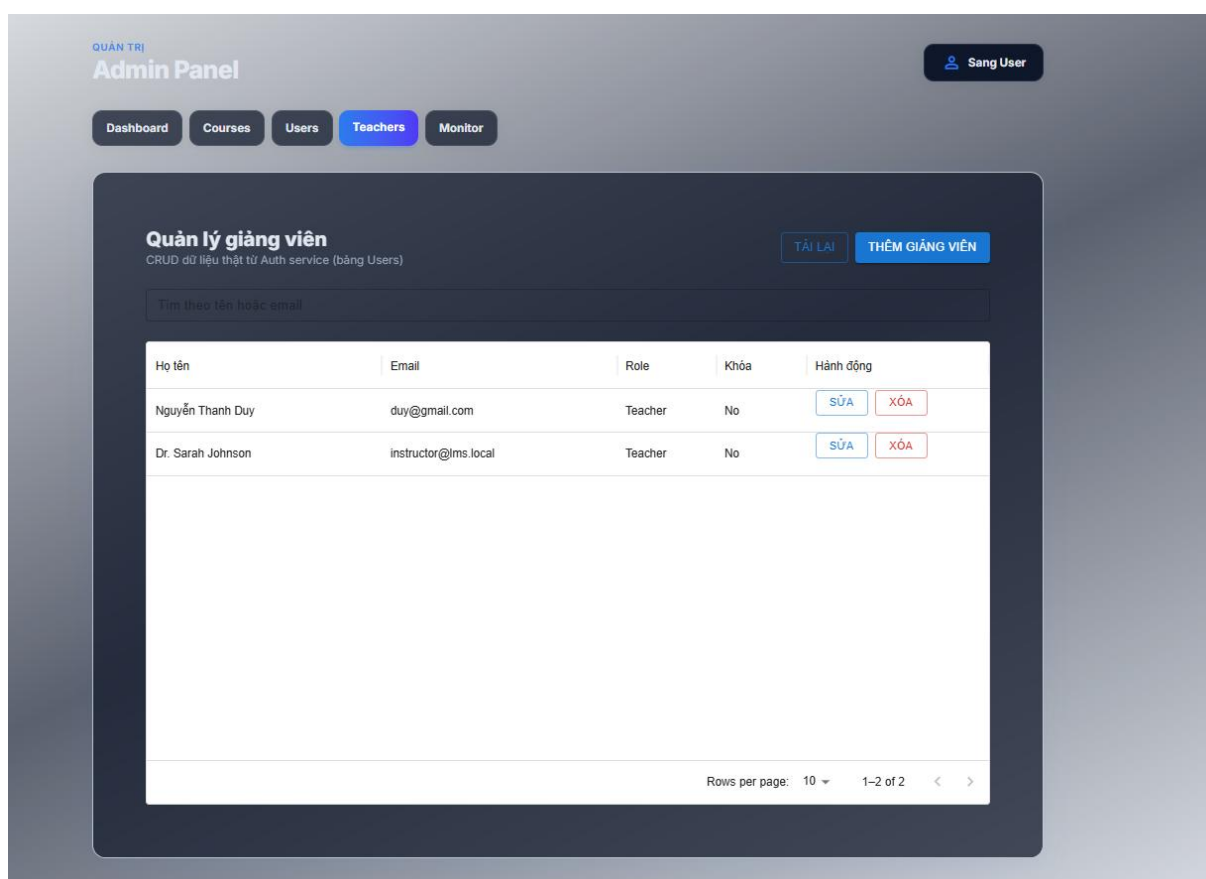
4.2.14. Giao diện quản lý người dùng



Hình 4.14. Giao diện quản lý người dùng

Giao diện Quản lý người dùng được thiết kế để tối ưu hóa việc kiểm soát và phân quyền toàn bộ tài khoản trên hệ thống từ Auth service. Dữ liệu học viên và giảng viên được trình bày trong một bảng biểu khoa học với các thông tin cốt lõi như: Họ tên, Email, Vai trò (Student/Teacher) và trạng thái tài khoản. Quản trị viên có thể dễ dàng thực hiện các thao tác quản trị trực tiếp thông qua bộ công cụ tích hợp bao gồm: tìm kiếm nhanh theo tên/email, lọc người dùng theo vai trò, thêm mới tài khoản và các nút hành động "Sửa/Xóa" tại từng dòng. Thiết kế này không chỉ đảm bảo tính minh bạch trong việc quản lý nhân sự mà còn giúp vận hành hệ thống một cách chặt chẽ, hỗ trợ xử lý kịp thời các vấn đề liên quan đến tài khoản người dùng.

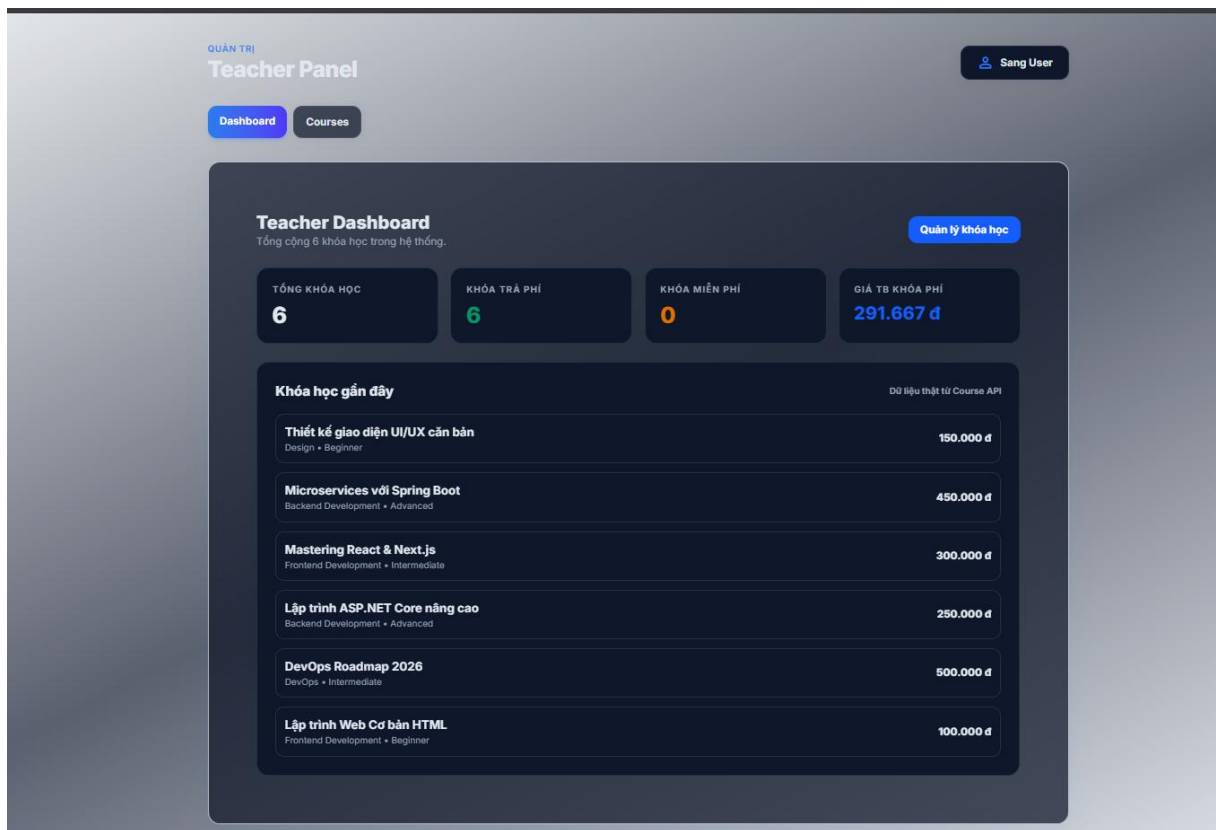
4.2.15. Giao diện quản lý giảng viên



Hình 4.15. Giao diện quản lý giảng viên

Giao diện Quản lý giảng viên được tối ưu hóa để quản trị riêng biệt đội ngũ nhân sự đào tạo trên nền tảng. Tương tự như cấu trúc quản lý người dùng, giao diện này tập trung vào việc hiển thị danh sách các tài khoản có vai trò "Teacher" với đầy đủ thông tin về họ tên và email định danh. Hệ thống cung cấp các chức năng quản trị mạnh mẽ như: thêm mới giảng viên, cập nhật thông tin và kiểm soát trạng thái hoạt động của từng cá nhân thông qua các nút hành động trực quan. Việc phân tách luồng quản lý giảng viên giúp quản trị viên dễ dàng thực hiện các nghiệp vụ chuyên biệt, đảm bảo dữ liệu đội ngũ giảng dạy luôn được cập nhật chính xác và đồng bộ từ dịch vụ xác thực hệ thống.

4.2.16. Giao diện quản lý của giảng viên



Hình 4.16. Giao diện quản lý của giảng viên

Giao diện Teacher Panel được thiết kế như một trung tâm quản lý cá nhân hóa dành riêng cho đội ngũ giảng dạy, tập trung vào việc thống kê và theo dõi hiệu suất các nội dung đào tạo. Khu vực biểu đồ tóm tắt phía trên cung cấp cái nhìn nhanh về tổng số khóa học, phân loại khóa học trả phí/miễn phí và giá bán trung bình, giúp giảng viên nắm bắt tình hình kinh doanh nội dung một cách tức thời. Phía dưới là danh sách các khóa học gần đây được lấy dữ liệu trực tiếp từ Course API, hiển thị chi tiết tên khóa học, danh mục, cấp độ và giá bán tương ứng. Với bố cục gọn gàng, tích hợp nút chuyển hướng "Quản lý khóa học" nhanh chóng, giao diện này hỗ trợ giảng viên tối ưu hóa quy trình vận hành và theo dõi sát sao lộ trình cung cấp kiến thức của mình trên nền tảng.

4.3. KIỂM THỬ HỆ THỐNG

Kiểm thử đơn vị (Unit Testing): Thực hiện kiểm tra các hàm xử lý nghiệp vụ cốt lõi trong từng dịch vụ (Auth Service, Course Service, Payment Service) để đảm bảo các logic tính toán giá tiền và xác thực dữ liệu đầu vào hoạt động đúng.

Kiểm thử tích hợp (Integration Testing): Kiểm tra khả năng giao tiếp giữa các Microservices thông qua API Gateway và sự đồng bộ dữ liệu giữa các cơ sở dữ liệu riêng biệt.

Kiểm thử chức năng (Functional Testing): Xác soát các luồng nghiệp vụ chính như: Đăng ký/Đăng nhập, thực hiện thanh toán qua VNPAY/MoMo, người dùng trả lời câu hỏi AI để hoàn thành bài học và quản trị viên cập nhật nội dung khóa học.

Kiểm thử giao diện (UI Testing): Đảm bảo tính nhất quán của chế độ Dark Mode, các hiệu ứng phản hồi người dùng (Glow effect, Progress bar) và khả năng hiển thị tương thích trên các trình duyệt phổ biến.

4.4. ĐÁNH GIÁ HIỆU NĂNG

Phân tích khả năng vận hành của hệ thống dựa trên các số liệu thực tế từ giao diện Monitor.

- Tính ổn định (Reliability): Hệ thống duy trì trạng thái "All Systems Normal" với độ trễ mạng thấp (khoảng 44ms), đảm bảo trải nghiệm học tập không bị gián đoạn.
- Tối ưu tài nguyên (Resource Optimization): CPU và RAM được phân bổ hiệu quả cho từng container dịch vụ. Các dịch vụ nhẹ như Auth API chỉ chiếm dụng tài nguyên tối thiểu, trong khi vẫn đảm bảo xử lý được lưu lượng truy cập lớn.
- Khả năng đáp ứng (Responsiveness): Việc tích hợp Trợ lý AI có tốc độ phản hồi nhanh (khoảng 0.5s sau khi người dùng ngừng gõ), tạo cảm giác tương tác mượt mà và thông minh.
- Tốc độ xử lý (Throughput): Hệ thống có khả năng xử lý đồng thời nhiều yêu cầu thanh toán và truy vấn dữ liệu khóa học nhờ vào kiến trúc phân tán.

4.5. ƯU ĐIỂM VÀ HẠN CHẾ

Ưu điểm:

- Kiến trúc hiện đại: Sử dụng Microservices giúp hệ thống dễ dàng mở rộng và bảo trì từng phần độc lập.
- Trải nghiệm người dùng (UX) vượt trội: Giao diện Dark Mode chuyên nghiệp, các tính năng Gamification (Bảng vàng) giúp tăng động lực học tập.
- Tính năng AI đột phá: Trợ lý AI cá nhân hóa và tính năng kiểm tra kiến thức bằng AI giúp nâng cao chất lượng đào tạo so với các LMS truyền thống.
- Quản trị toàn diện: Dashboard quản trị cung cấp đầy đủ công cụ từ giám sát hạ tầng kỹ thuật đến quản lý doanh thu và nhân sự.

Hạn chế:

- Độ phức tạp: Việc vận hành nhiều dịch vụ Microservices đòi hỏi hạ tầng server mạnh và quy trình triển khai (CI/CD) phức tạp hơn.
- Dữ liệu AI: Hiệu quả của trợ lý AI phụ thuộc lớn vào tập dữ liệu đào tạo và độ chính xác của các mô hình ngôn ngữ lớn (LLM).

KẾT LUẬN

1. Đóng góp đề tài

Đề tài Intelligent LMS xây dựng một hệ thống quản lý học tập hiện đại dựa trên kiến trúc Microservices (.NET 8) kết hợp trí tuệ nhân tạo, mang lại giải pháp giáo dục trực tuyến linh hoạt và có khả năng mở rộng cao. Điểm nhấn của dự án nằm ở việc tích hợp sâu AI (Python, FastAPI) để cá nhân hóa lộ trình học tập, tự động đề xuất khóa học và dự báo nguy cơ bỏ học thông qua việc xử lý dữ liệu thời gian thực bằng Apache Kafka. Với sự kết hợp đa nền tảng công nghệ cùng quy trình triển khai chuyên nghiệp dựa trên Docker, đề tài không chỉ tối ưu hóa hiệu năng quản trị mà còn giải quyết triệt để bài toán về sự tương tác và gắn kết người dùng, định hướng trở thành một sản phẩm EdTech thực tế hoàn chỉnh trong kỷ nguyên chuyển đổi số.

2. Hướng phát triển tương lai

- Mở rộng hệ sinh thái AI: Phát triển tính năng AI tự động tạo đề thi và đánh giá bài tập thực hành (coding/essay) của học viên một cách chi tiết hơn.
- Ứng dụng Mobile: Phát triển phiên bản ứng dụng di động (Android/iOS) đồng bộ với phiên bản Web để người dùng có thể học mọi lúc mọi nơi.
- Hệ thống tương tác cộng đồng: Tích hợp tính năng livestream giảng dạy trực tuyến và diễn đàn thảo luận real-time giữa giảng viên và học viên.
- Tối ưu hóa Big Data: Áp dụng phân tích dữ liệu lớn để dự đoán xu hướng học tập và đưa ra các tư vấn nghề nghiệp chính xác cho người dùng dựa trên lịch sử học tập.

TÀI LIỆU THAM KHẢO

- [1] „Learning Management System,” *International Journal of Information Technology and Computer Engineering*, 2025.
- [2] V. Bradley, „Learning Management System (LMS) Use with Online Instruction,” *International Journal of Technology in*, 2020.
- [3] N. & K. F. Kasim, „Choosing the Right Learning Management System (LMS) for the Higher Education Institution Context: A Systematic Review,” *Int. J. Emerg. Technol. Learn.*, vol. 11, pp. 55-61, 2016.
- [4] P. & S. J. Veluvali, „Learning Management System for Greater Learner Engagement in Higher Education—A Review,” *Higher Education for the Future*, vol. 9, pp. 107 - 121, 2021.
- [5] K. Thangavel, „Learning Management Systems (LMS) in Higher Education: Enhancing Teaching, Learning, and Administrative Processes,” *Thiagarajar College of Preceptors Edu Spectra.*, 2024.
- [6] A. & M. G. Maluleke, „Examining the Impact of Learning Management Systems in African Higher Education: A Systematic Review,” *African Journal of Inter/Multidisciplinary Studies.*, 2025.
- [7] M. & H. M. Munna, „Digital Education Revolution: Evaluating LMS-based Learning and Traditional Approaches,” *Journal of Innovative Technology Convergence.*, 2024.
- [8] J. T. S. & T. S. Satmune, „Study and analysis of the need for an intelligent lesson content management system for undergraduate digital technology education,” *International Journal of Innovative Research and Scientific Studies*, 2025.
- [9] Y. G. I. & Z. V. Artamonov, „se analysis of microservices in e-learning system with multi-variant access to educational materials,” *Technology audit and production reserves.*, 2021.
- [10] F. H. A. H. D. & W. N. Dahri, „Implementation of Microservices Architecture in Learning Management System E-Course Using Web Service Method,” 2022.
- [11] H. Y. S. G. M. & Y. M. Gu, „Research on online teaching platform system based on microservice architecture,” *MATEC Web of Conferences.*, 2022.

- [12] M. & K. M. Demydenko, „ONLINE PLATFORM PROTOTYPE USING MICROSERVICE ARCHITECTURE AND CONTAINERIZATION FOR DIGITALIZATION OF THE EDUCATIONAL PROCESS,,” *Системи управління, навігації та зв'язку. Збірник наукових праць.*, 2023.
- [13] A. V. A. & N. A. Obukhov, „Microservice Architecture of Virtual Training Complexes,” *Informatics and Automation.*, 2022.
- [14] O. G. M. K. M. & G. A. Arsirii, „INFORMATION TECHNOLOGY OF SUPPORTING ARCHITECTURAL SOLUTIONS USING POLYGLOT PERSISTENCE CONCEPT IN LEARNING MANAGEMENT SYSTEMS,” vol. 3, pp. 13-31.
- [15] G. O. A. & P. A. Blinowski, „Monolithic vs. Microservice Architecture: A Performance and Scalability Evaluation,” *IEEE Access*, vol. 10, pp. 20357-20374, 2022.
- [16] F. M. M. F. W. A. H. F. E. & T. T. Tapia, „From Monolithic Systems to Microservices: A Comparative Study of Performance,” *Applied Sciences.*, 2020.
- [17] A. J. U. J. I. & M. N. I.Y, „DEVELOPMENT OF AN AI-DRIVEN ADAPTIVE LEARNING MANAGEMENT SYSTEM USING DATA ANALYTICS,,” *International Journal of Scientific Research in Modern Science and Technology.*, 2025.
- [18] F. K. M. T. M. A. A. & A. S. Naseer, „Integrating deep learning techniques for personalized learning pathways in higher education,,” *Heliyon*, vol. 10, 2024.
- [19] P. Chitrakar, „Monolith vs. Microservices: A Comprehensive Analysis of Architectural Evolution in Software Development,” *International Research Journal of Modernization in Engineering Technology and Science.*, 2025.
- [20] M. A. Y. S. J. S. F. & U. M. Murtaza, „ AI-Based Personalized E-Learning Systems: Issues, Challenges, and Solutions,” *IEEE Access*, vol. 10, pp. 81323-81342, 2022.
- [21] R. S. M. Y. M. V. R. & S. S. Divya, „Automating Education: AI-Powered LMS for Personalized Learning and Consistent Evaluation using BERT,,” *International Conference on Self Sustainable Artificial Intelligence Systems (ICSSAS)*,, nr. 3, pp. 1533-1540, 2025.

- [22] Y. K. I. & N. L. Suhandi, „RANCANGAN SISTEM PEMBELAJARAN DARING BERBASIS WEB.,” *JRIS: JURNAL REKAYASA INFORMASI SWADHARMA*, 2021.
- [23] Ethelbert, Obinna, et al, „A JSON token-based authentication and access management schema for cloud SaaS applications.,” *IEEE 5th international conference on future internet of things and cloud (FiCloud)*, 2017.
- [24] Nwabuwe, Augustine, et al, „Fraud mitigation in attendance monitoring systems using dynamic QR code, geofencing and IMEI technologies.,” *International Journal of Advanced Computer Science and Applications*, vol. 14, pp. 938-945, 2023.
- [25] Chu, Thai Son, and Mahfuz Ashraf., „Artificial intelligence in curriculum design: A data-driven approach to higher education innovation.,” *Knowledge*, vol. 14, nr. 5, 2025.
- [26] Nagy, Marcell, and Roland Molontay, „Interpretable dropout prediction: Towards XAI-based personalized intervention.,” *International Journal of Artificial Intelligence in Education*, vol. 34, pp. 274-300, 2024.
- [27] Li, Peng, Cheng Che, and Rui Hou. , „Nacc-Guard: a lightweight DNN accelerator architecture for secure deep learning: P. Li et al.,” *The Journal of Supercomputing*, vol. 80, pp. 5815-5831, 2024.
- [28] Ribeiro, Murilo B., and Kelly R. Braghetto, „A scalable data integration architecture for smart cities: implementation and evaluation.,” *Journal of Information and Data Management*, vol. 13, 2022.
- [29] M. B. F. & O. T. Aparicio, „An e-learning theoretical framework.,” *Educational Technology & Society*, vol. 19, nr. 1, p. 292–307, 2016.
- [30] S. e. a. Zhou, „Improving performance of web applications with in-memory caching: A case study of Redis,” *IEEE 3rd International Conference on Cloud Computing and Big Data Analysis (ICCCBDA)*, p. 393–397, 2018 .
- [31] D. e. a. Raho, „On the performance of microservices-based cloud applications: Container orchestration and resource management.,” *IEEE International Conference on Cloud Engineering (IC2E)*, 2019.
- [32] S. Newman, „Building Microservices: Designing Fine-Grained Systems.,” *O'Reilly Media.*, 2015.

- [33] V. (. Chirukuri, „Building an End-to-End Reconciliation Platform for Accurate B2B Payments in New-Age Fintech Distributed Ecosystems: A Case Study using Microservices and.,” *European Journal of Computer Science and Information*, 2025.
- [34] A. & F. E. Nurdiansyah, „OPTIMIZING DATA CONSISTENCY IN MICROSERVICE ARCHITECTURE USING THE SAGA PATTERN AND EVENT-DRIVEN APPROACH,” *INTECOMS: Journal of Information Technology and Computer*, 2025.
- [35] F. A. T. A. F. & Z. A. Eyvazov, „Beyond Containers: Orchestrating Microservices with Minikube, Kubernetes, Docker, and Compose for Seamless Deployment and Scalability.,” *International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO)*, pp. 1-6, 2024.
- [36] M. R. J. F. J. A. S. S. N. & C. Y. Alam, „Orchestration of Microservices for IoT Using Docker and Edge Computing,” *IEEE Communications Magazine*, vol. 56, pp. 118-123, 2018.
- [37] R. J. W. & K. D. Xu, „Microservice Security Agent Based On API Gateway in Edge Computing,” *Sensors (Basel, Switzerland)*, p. 19, 2019.
- [38] C. D. K. Z. S. W. H. H. S. & L. J. Wei, „Enhanced Recommendation Systems with Retrieval-Augmented Large Language Model,” *J. Artif. Intell. Res*, vol. 82, pp. 1147-1173, 2025.
- [39] S. Dhawan, „Online learning: A panacea in the time of COVID-19 crisis,” *Journal of Educational Technology Systems*, vol. 49, nr. 1, p. 5–22, 2020.
- [40] D. J. M. M. R. & S. J. Al-Fraihat, „Evaluating E-learning systems success: An empirical study,” *Computers in Human Behavior*, vol. 102, p. 67–86, 2020.
- [41] H. J. R. & B. G. Coates, „A critical examination of the effects of learning management systems on university teaching and learning.,” *Tertiary Education and Management*, vol. 11, nr. 1, p. 19–36, 2005.
- [42] Ratican, J., Hutson, J., & Wright, A., „ A proposed meta-reality immersive development pipeline: Generative ai models and extended reality (xr) content for the metaverse,” *Journal of Intelligent Learning Systems and Application*, vol. 15, 2023.